SAPIENZA
UNIVERSITÀ DI ROMA

Explainable Inference on Sequential Data via Memory-Tracking

Facoltà di Ingegneria dell'informazione, informatica e statistica
Corso di Laurea Magistrale in Artificial Intelligence and Robotics

Candidate
Biagio La Rosa
ID number 1389823

Thesis Advisor
Roberto Capobianco

Academic Year 2019/2020

Explainable Inference on Sequential Data via Memory-Tracking
Master's thesis. Sapienza – University of Rome

© 2020 Biagio La Rosa. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: biagiomattialarosa@gmail.com

A

Abstract

Recent progress in deep learning is revolutionizing many domains, increasing the spread and impact of Artificial Intelligence on our daily life. The improvements are achieved using increasingly complex networks, characterized by a myriad of small units highly connected that cooperate to reach a common goal.
The complexity comes at the cost of interpretability. Indeed, the persons see them as *black boxes*, where you feed the data into the network and collect the response, without the possibility to unfold the steps that bring the model to return a specific answer.

To overcome these limits the subfield of AI called eXplainable Artificial Intelligence (XAI), has gained a new interest. It aims to provide highly interpretable models or tools able to explain, at least partially, why a black box takes a decision. Although there exist many works covering tabular data and single-step predictions, a handful can deal also with sequential data.

This work of thesis proposes an extension to Memory-Augmented Neural Networks (MANNs) to get hints of explanations for sequential problems, namely a new Explanation Module. Tracking the reading and writing mechanisms of the memory, and connecting them both to the input sequence and the network output, the module allows to extract insights about the decision process.

The system is tested on two tasks: a modified version of *T-maze*, a control task to evaluate the algorithm's ability to correlate events far apart in history, and *Story Cloze Test* (SCT), which is a commonsense reasoning task for evaluating the language understanding. The experiments show that the module is able to highlight the correct decision process in the first task, and reconstruct the most relevant sub-sequence used by the network in the latter. Furthermore, the adequacy of explanations is assessed against two recent tests, which confirm that the proposed method depends correctly on both the labels and the learned parameters.

Contents

1 Introduction **9**

2 Background **11**

2.1 Machine Learning 11

2.1.1 Supervised learning 12

2.1.2 Unsupervised learning 12

2.1.3 Reinforcement learning 13

2.2 Deep Learning 13

2.2.1 Neural Networks 13

2.2.2 Recurrent Neural Networks 18

2.2.3 Memory-Augmented Neural Networks 20

2.2.4 Deep Q-Learning 25

2.3 Explainable Artificial Intelligence 26

2.3.1 The problem of evaluation 28

3 State of The Art Approaches **31**

3.1 Global Approximators 31

3.2 Local Surrogate Models 33

3.3 Set of Rules 34

3.4 Backpropagation-based methods 35

3.5 Saliency Maps 36
3.6 Counterfactual Explanations 37
3.7 Rigorous Global Explanations 38

4 Methodology **39**
4.1 Problems of interest 39
4.1.1 The Story Cloze Test 39
4.1.2 T-maze 40
4.2 Simplified DNC 41
4.3 Explanation Module 43

5 Experiments and Results **47**
5.1 Configurations 47
5.2 T-maze 47
5.2.1 Representation 47
5.2.2 Experiments 48
5.2.3 Results 48
5.3 Story Cloze Test 49
5.3.1 Representation 49
5.3.2 Experiments 49
5.3.3 Results 51
5.4 Novelty and relation with other SOTA approaches 56

6 Conclusion **59**

Chapter 1

Introduction

The field of Artificial Intelligence (AI) is the field of Computer Science that aims to develop machines able to solve tasks that typically require human intelligence, simulating it and taking rational and autonomous decisions on complex problems. The first scientific works on the subject come from 60 years ago, but only recently it has been widely adopted to tackle everyday tasks, thanks to the use and spreading of more complex machine learning models, called *deep learning models*.
A machine learning model is an algorithm that performs a task without being explicit programmed to solve it, learning a mathematical model that captures the common patterns inside a set of data.
The huge amount of data, available today, combined with more powerful computer systems and new methodologies, allowed to train bigger and more advanced models able to solve real-world problems.

Today the AI impacts our life in many aspects, ranging from the vocal assistant on our phone to online advertising systems, from the purchase and sale on the trade markets to autonomous drive. The success and performance of complex approaches bring with them an increasing opacity of the architectures and the rationale behind their decisions. The more powerful deep learning models, indeed, contain thousand of small units, highly interconnected, and millions of parameters, that make difficult, or impossible, to reconstruct the exact series of steps that the model has taken to produce the final decision. For this reason, many safety and mission critical domains, like health care or politics, hold out to use older machine learning models, more interpretable but less powerful, or they don't use AI at all.

In fact, when a machine decision can significantly change the life of a person, you have to be sure that the decision is the right one, understanding the reasons behind it. Moreover, Bolukbasi et al. [12] proved that a model can learn some stereotypes from the data and they can misguide a decision, discriminating people only on the basis of them.
Taking care about these problems, the European Union's law on "right of explanation" [30] establishes that the citizens have the right of explanations about the

algorithmic decision that was made affecting them.

On top of that, the poor understanding of the decision process, puts a brake on the progress of the field itself. Several approaches work very well in some domains, but have poor performance in others, leaving us with a hypothesis about such behavior that cannot be proven. The debug of a developing model is very hard, under these constraints, and the most used techniques are grid search over parameters and trial-and-error for the possible configurations of the model.

For all these reasons, recently, the subfield of AI called eXplainable Artificial Intelligence (XAI) has gained a new interest. It tries to provide tools able to explain, at least partially, an artificial intelligent agent decision, or models that focus both on performances and interpretability of their decisions.
The thesis is located under the umbrella of XAI, trying to provide hints of explanations about the decisions taken by a sequential deep learning model. In particular, it describes a module for a Memory-Augmented Network - Differentiable Neural Computer - that, keeping track of the memory usage during the inference phase, it is able to reconstruct the most relevant sub-sequence of input, allowing to get hints of explanations on sequential problems.

The manuscript consists of six chapters, including this one: the second chapter describes the basic concepts that will be used in the successive chapters, presenting the main architectures of AI and the problem of explainability. The third chapter focuses the attention on the eXplainable Artificial Intelligence, describing the state of the art approaches, summarizing their weakness and strengths. The fourth chapter describe the system in all its components, the steps to get explanations and the considered tasks. The chapter five presents the experiments made and their results, along a discussion about them. Finally, the last chapter summarizes the insights obtained from the work and the possible future developments.

Chapter 2

Background

2.1 Machine Learning

Machine Learning (ML) is a field of AI that aims to teach a computer or algorithm to perform some specific tasks without being programmed to do them. This ability is very useful for several applications: in space exploration allows the probes to have a bit of autonomy, relaxing the need of slow communications with the operational bases; in robotics the robot can handle the uncertainty of new environments; in self-driving car it helps to monitor all the around objects and so on.

To acquire such an ability, the algorithm learns to do something observing *data*, that act as past experiences or background knowledge of the given task. The set of data used to *train* an algorithm is called *dataset*, while the process of learning from them is called *training process*. During the process the algorithm builds a mathematical model that, at the end, should be able to deal with new instances of the task, never seen before, in a satisfactory way.

There are several key factors crucial to get good models. For example, a representation able to capture, numerically, the similarities and the differences between two described objects, can have a huge positive impact on the goodness of the training process. Others are the set of mathematical tools used by the algorithm to improve its performance on the task (*optimizations*), the *objective function* that fixes the numerical quantity to be optimized, and the *evaluation scores* used to assess the quality of improvements.

On the basis of the way in which the computer learns, there can be distinguished three paradigms of learning: *supervised learning*, *unsupervised learning* and *reinforcement learning*.

2.1.1 Supervised learning

Supervised learning is the most used and studied type of learning. The name supervised comes from the fact that in these settings there is a teacher that tells the model if its behavior is the correct one or not.
The teacher is represented as a set of desired answers, called *labels*, associated with each instance of the dataset. Therefore, the algorithm, during the training process, should learn the function f that maps the inputs x to the correct label l.

$$l = f(x) \tag{2.1}$$

During the training process, the model tries to guess the correct answer and then checks if its prediction is equal to the linked label or it has to change its behavior to remedy to the mistakes.

The supervised paradigm fits well two types of task:

- classification problems, where the goal is to predict the class of an object among a limited set of predefined classes;
- regression problems, where typically the goal is to predict a future value, given a set of past and present observations. In regression the output is unbounded and can assume any value.

2.1.2 Unsupervised learning

In unsupervised learning it is removed the need of labels and the only required source of knowledge is the dataset. The absence of labels is a big advantage of the approach, making it potentially applicable to any source of knowledge ranging from the data sensors to the information retrieved from the web.
The algorithm has to learn by itself how to find the patterns and structures in the data.

The main problems addressed by unsupervised learning are:

- dimensionality reduction, where the goal is to cut the dimensions of the data representation, extracting the most useful features, and ignoring the others;
- clustering, where the algorithm has to group the instances of the dataset, on the basis of the discovered patterns, to extract algorithmic relationships;
- density estimation, that aims to infer an a priority probability distribution that could generate the observed data.

2.1.3 Reinforcement learning

In reinforcement learning, the computer learns interacting with an environment. Basically, it tries to do something and receives a feedback from the environment: if the feedback was positive, the algorithm is encouraged to do the action again, otherwise it will likely try another action next time. Therefore the feedback process acts as a teacher that suggests if the action was good or bad to accomplish its goal.

More formally, let be the environment in a state s. The agent can perform an action chosen from a set of possible ones $\{a_1, a_2, ...a_n\}$: each of them causes an environment transition from the current state to another state s'. The transition can be fixed or stochastic, meaning that it can lead the environment to different states, each with its own probability. In general the transition is Markovian, depending only on the current state instead of the whole history of past states.

Given a couple of an action a and a current state s the environment sends a reward $r(s_t, a_t)$ to the agent. The goal of the agent is to maximize the total rewards gained over an episode, where an *episode* is a series of actions and states $\{s_1, a_1, s_2, a_2....s_t, a_t\}$.

Therefore, the agent must learn a *policy* $\pi(s)$ that maps the state observations to a probability over actions. In particular, it is desirable that it learns the *optimal policy* π^*, the policy that yields the highest expected reward.

Reinforcement learning is applied in multiple problems, covering robot navigation, games, real-time decisions and learning tasks.

2.2 Deep Learning

2.2.1 Neural Networks

Deep Learning is the last frontier of Machine Learning. The models and algorithms used in Deep Learning are more complex that traditional Machine Learning models and take inspiration from the biological processes of human and animal brains.

The first works on neural networks, the architectures behind the Deep Learning, started from the observation that the human brain is faster, of several orders of magnitude, than computer programs to solve complex problems, thanks to its conglomerate structure of *neurons*. They are organized in an, unknown, efficient way that makes vision, motion control, pattern recognition and other tasks solvable in milliseconds. The aim of these earlier works was to try to build a neural network of artificial neurons that behaves in a similar way and has similar nice features.

The first mathematical model of an artificial neuron was published in 1943 by McCulloch & Pitts [50] and it represents essentially a binary classifier that decides

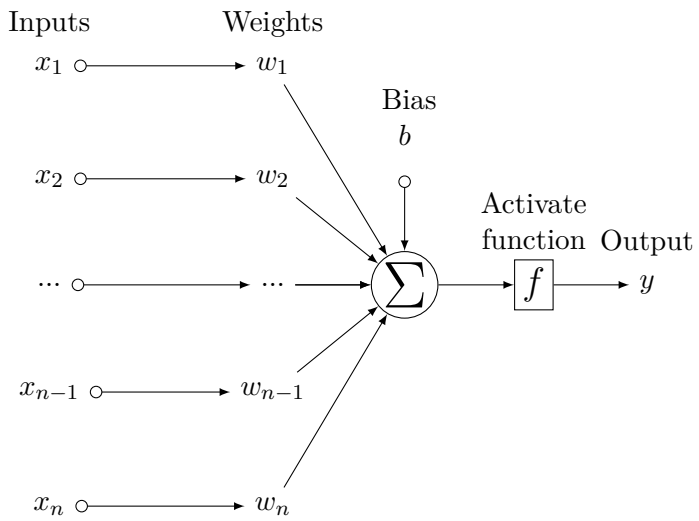

Figure 2.1. A representation of a artificial neuron. x_i are the inputs of neuron, each weighted by the weight w_i. The sum, of each product $x_i w_i$ and the bias term b, is the input of an activation function that returns the output of neuron.

whether an input belong to a specific class.

The neuron is a unit that, in its simplest form, shown in fig 2.1, takes as input n real numbers $x_i \in \mathbb{R}$, performs operations on it and returns a binary output. Each input x_i is multiplied with an associated weight $w_i \in \mathbb{R}$, then the neuron computes the linear combination of them, adds a constant term called *bias* b and compares the result, obtained so far, against a fixed threshold using a function called *activation function*.

The $\mathbf{x} = [x_1, x_2, ..., x_n]$ vector represents the features of an object that helps to differentiate it from other similar objects. For example, suppose that we want to separate cats and spiders. Then the features of the input object, that describe the current animal, could be the number of legs, the height, the color and so on.

The weights $\mathbf{w} = [w_1, w_2, ..., w_n]$ regulate the importance of each feature for the classification: a higher magnitude of w_i, respect the other weights, means that the feature x_i contributes more than another to predict the right class. In the previous example, a valid choice can be to fix a high weight for the number of legs, because, in general, the cats have four legs and spiders eight, so that feature is very useful and strong to classify correctly the current object. Conversely the color could have a low weight, because there exist both spiders and cats of different colors.

Therefore, the output of the neuron can be expressed by the mathematical formula:

$$y = \phi \left[\left(\sum_{i=1}^{n} w_i x_i \right) + b \right] \tag{2.2}$$

where ϕ is the activation function that modifies the behavior of the neuron. In the

cited work they use, as activation function, the *binary step function* that fires the neuron if the sum is above a fixed threshold and returns 1 as output, otherwise the neuron remain inactive and return -1. The step function can be described by the equation:

$$\phi(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases} \tag{2.3}$$

Finally, in order to understand how good is the prediction, it should be compared against the desired answer through a function called *loss function* $E(n)$ or error function. The simplest example of loss function is the absolute value of the difference between them:

$$E_n = |y_n - y_{d,n}| \tag{2.4}$$

One of the main weaknesses of the previous neuron model is that it requires to fix, a priori, the weights, using the knowledge about the problem. Starting from this observation, Rosenblatt in 1957 proposes the perceptron [59], a learning algorithm that allows to adapt the weights iteratively, in a process called *training process*, making the neuron able to learn by itself what features are more important to make the correct prediction.
The rule to update the weights is called *error-correction learning rule* and can be expressed as:

$$w_{i,t+1} = w_{i,t} + \eta E_{j,t} x_{i,t} \tag{2.5}$$

where $w_{i,t}$ is the value of weight i at iteration t, $E_{j,t}$ is the error respect to the desired output at iteration t. Note that, if the prediction is correct, then $y_{d,j} = y_{j,t}$, therefore $E_j = 0$ and the weights aren't updated. Finally, the $\eta \in [0, 1]$ parameter is the *learning rate* that controls the size of the update: a small value will make small adjustments averaging the past inputs, while a large value will produce fast adaptation, but taking in less consideration the past mistakes.

The nice feature of the perceptron is that it converges after a finite number of iterations n_0, provided that the considered classes are linearly separable. Two classes are linearly separable if it exists a combination of values for the weight vector $\mathbf{w}$ such that $\mathbf{w}^T\mathbf{x} \geq 0$ holds for every input vector $\mathbf{x}$ belonging to the first class and $\mathbf{w}^T\mathbf{x} < 0$ holds for the other inputs. The convergence guarantees that the neuron won't make mistakes after the iteration n_0 and the weights won't be updated anymore.

The limitations of the perceptron were surpassed with the introduction of some concepts: the activation function and loss function in the form of differentiable functions, the backpropagation algorithm and the multilayer perceptron. The combination of these concepts is the foundation of deep learning and the base for all the state-of-the-art approaches.

The *multilayer perceptron* consists of interconnected neurons, organized in layers, transferring information to each other, as shown in Figure 2.2. Each circle in the image is an artificial neuron, like the one described before. The input layer represents the features $\mathbf{x}$, so it basically returns the input vectors, while the output layer

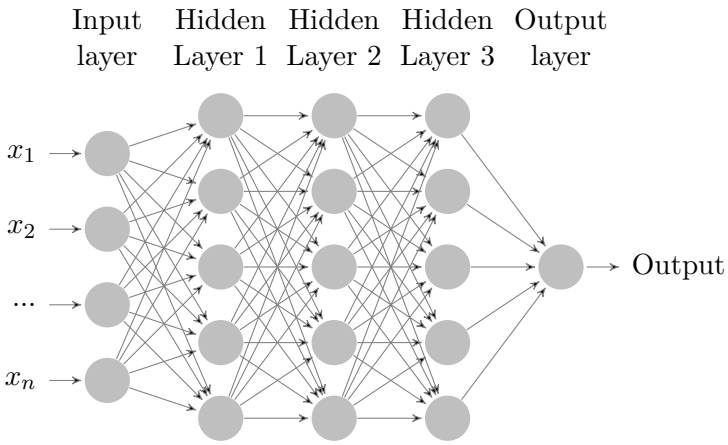

Figure 2.2. An example of 3-layer perceptron. Each node in the hidden layers is connected with all the nodes of the previous layer and all the nodes of the next layer.

collects the outcomes of the last layer and takes the final decision. Each neuron in a hidden layer i takes, as input, all the outputs of the previous layer nodes, weighted by the layer weights $\mathbf{w_i}$, computes its output and send it to all the neurons of the next layer.

The highly connected structure allows each neuron to become specialized in the detection of a particular correlation between features, making the whole network able to represent any function, given the right weights choice. The name Deep Learning Models refers to the organization in layers, each of which with many nodes, that makes the model "deep".

The algorithm to update the weights of multilayer perceptron adaptively, during the training process, is the *backpropagation algorithm* [60]. It requires that the activation function ϕ is differentiable - a function whose derivative exists at each point in its domain and that can be drawn without any breaks, bends or cups. The step-function used before isn't differentiable, therefore it can be replaced by one between sigmoid function, tanh function (shown in Figure 2.3), the most recent ReLU [3], or finally ELU [18] or Leaky ReLU that are differentiable only on specific intervals.

The algorithm proceeds in two phases: in the forward phase the input is propagated through the network till the output layer and no changes are made; at the output layer it is computed the error using the loss function. Note that this time the loss isn't the simple difference between the desired output and the response of the network, but usually is squared or anyway differentiable; lastly, the error is propagated from the output layer to the input layer in the backward phase adjusting the weights of each layer.

Each neuron must be designed to compute both its output of the forward phase and the gradient vector for the backward phase. The gradient is the mathematical

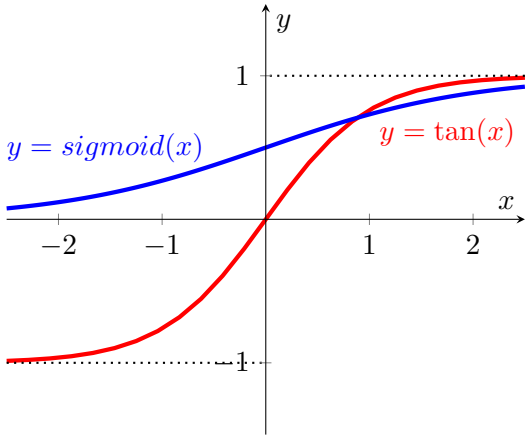

Figure 2.3. Examples of differentiable activation functions. The blue line is the sigmoid function that return a value in the range [0,1], while the red line describes the tanh function that outputs values in the range [-1,1].

tool that allows to adjust the neuron weights in accordance to their contribution to the outcome of that neuron. While for the output neuron the error-correction rule can be applied, the same can't be done for the hidden neurons. In fact, there isn't a clear desired output for such nodes, and it should be determined in terms of the errors of all the neurons to which the node is connected.

The algorithm can be summarized by the following equations.
Let be $a_j = \sum_i w_{ji} z_i$ the linear combination of input values and input weights for the hidden neuron j, $z_j = \phi(a_j)$ the output value of the neuron j returned by the activation function ϕ and $E_n(\mathbf{w})$ the error function for the current example n. For output nodes the error can be, for example:

$$\delta_n = \frac{1}{2}(y_n - y_{d,n})^2 \tag{2.6}$$

We then backpropagate the error, calculating for each neuron j in the previous layer the quantity:

$$\delta_j^L = \phi'(a_j) \sum_k w_{jk} \delta_k^{L+1} \tag{2.7}$$

where the superscript indicates the layer of the neuron, δ_k^{L+1} are the error of neurons of the layer L + 1 linked to the current neuron with the weight w_{jk} and ϕ' is the derivative of the activation function of the node j. Now, multiplying the error of the neuron for the input received from the node i in the previous layer, we obtain the gradient of the error respect the weight w_{ji}

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j^L z_i^{L-1} \tag{2.8}$$

The set of the partial derivatives for all the weights in the network forms the gradient vector ∇E_n, or in other words, the changes that must be made to the weights for improving its performance on the current example. The weights are updated with the general formula:

$$w^{t+1} = w^t - \eta \nabla E_n \tag{2.9}$$

The way in which you can combine the gradient, the learning rate and the error function, to produce the update equation for the weights, allow the born of different algorithms, called *optimizers*, like Adam [43], RMSProp, AdaGrad [24], etc.

While, as shown, these elements allow us to solve or approximate more complex problems, they are also the responsible for the deficiencies in our knowledge of the behavior of the network. The high connectivity, the employment of layers and the nonlinearity of activation function, make the theoretical analysis difficult and make them similar to *black boxes*, where you feed the data and receive a response without motivation about it.

2.2.2 Recurrent Neural Networks

The models presented so far see the features as independent each other and they don't take account of the temporal and spatial correlation between them. To overcome this limitation there are developed Convolutional Neural Networks (CNNs) for the spatial relations and Recurrent Neural Networks (RNNs) for temporal relations.

Suppose you have to annotate the words in the sentence shown below, with their part of speech tag (verb, noun, etc.).

```
We're holding them because they refuse to tell us the truth.
```

Some of the words, like the pronoun "they", are easy to solve and they can be annotated without seeing the context around them. Consider instead the ambiguous word "refuse": it can be a verb with the meaning of «to decline» or a noun indicating «something that is discarded»; its tag depends by the sequence of previous words. Because the preceding word is a pronoun, it is likely that the correct tag is the verb. This is an example of *sequential problem* where you have a sequence, whose elements are temporally related.

A Recurrent Neural network (RNN) is a type of network that aims to solve this kind of problems. In a RNN the neurons process each item of the sequence recursively, maintaining an internal state h_t through the sequence and emitting an output y_t for each element.
If the recurrent unit is unfolded, like shown in Figure 2.4, it can be viewed that, the recurrent invocation of the neurons, forms a directed graph along the temporal sequence. At each time-step t the neuron takes the internal state of the previous step h_{t-1}, returns its response y_t and send its internal state h_t to the next time-step. Regarding the update of weights, in RNN it is used a variant of backpropagation, called Backpropagation Through Time (BTT), that unrolls the RNN and applies the propagation starting from the last time-step. Note that here the error of each time-step can be easily computed as the difference between the desired answer and the current output, but the gradient — the partial derivatives — depends on all the previous time-steps. In the end, the time-step errors are accumulated, and the usual optimization is performed.

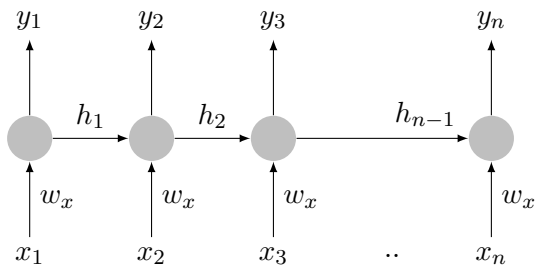

Figure 2.4. An unfolding view of a RNN cell. At each time-step t the cells read the input value x_i and return an output y_i, maintaining an internal state h_i used in the next time-step $t+1$.

The main problem of these basic architectures is the *vanishing gradient problem* [35]. When the sequence is longer than a few steps, the gradient becomes smaller and smaller from the last step to the first one and can vanish in the process, preventing the weights from changing their values. The problem can happen also for Feed-Forward Neural Network with very deep architectures, but clearly it happens more often in the recurrent networks. The result is that the first steps of the sequence are ignored by the training process, becoming useless very soon.

In order to mitigate its impact and other problems, the basic structure of the recurrent unit has been extended and exploited, creating new types of recurrent neurons, like Long Short-Term Memory (LSTM) [36] and Gated Recurrent unit (GRU) [22].

The Long Short-Term Memory networks (LSTM) [36], for example, use a memory cell as neuron. Each memory cell, for each time-step, takes as input the current element x_t in the sequence, the previous output h_{t-i} and the previous cell state c_{t-1}. The key element is the cell state, that allows to store information about the time-steps for longer periods respect the basic RNN unit.
Two sigmoid layers, called gates, manage the content of the cell state. The *forget gate* f_t decides what information throw away from it, and the *input gate* i_t decides which values of the cell state to update.
The new information, to be stored by the input gate, is computed by a tanh layer that takes as input x_t and h_{t-1} and returns the new candidate values $\hat{C}_t$.
Therefore the current cell state is computed as:

$$C_t = f_t C_{t-1} + i_t \hat{C}_t \tag{2.10}$$

Finally, the output is obtained from the multiplication of the *output gate* o_t, that decides how much of the current cell state will contribute to the current output respect to the previous one, and the current input. Notice that the cell state is filtered by a tanh layer before to be passed along the output gate to get the final output.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \tag{2.11}$$

$$h_t = o_t tanh(C_t) \tag{2.12}$$

The problem of vanishing gradient is reduced thanks to the use of cell state, allow-

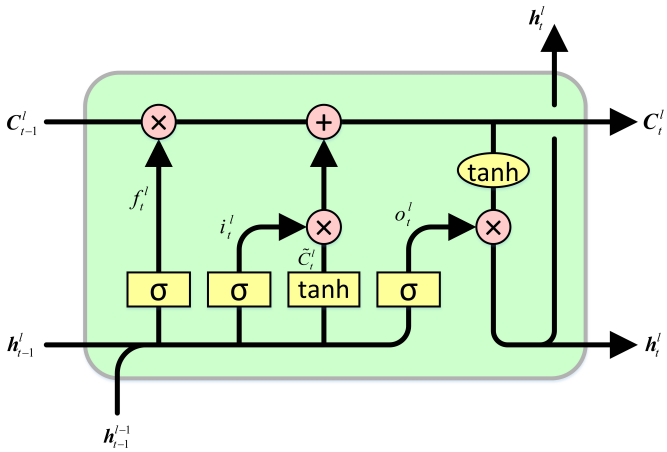

Figure 2.5. An overview of LSTM cell. Yellow box are layer transformation, where tanh indicates the application of tanh function, while σ describes a sigmoid layer. The red circles describe operations like sum and product of vector.

ing also the earlier steps to influence the prediction of the later steps.
LSTMs are nowadays widely applied to several problems, reaching great performance, and many variants of them have been proposed to mitigate some of their weaknesses. Bidirectional LSTMs [38], for example, allow the network to exploit the context, both from the left side and right side of the element in the sequence.

2.2.3 Memory-Augmented Neural Networks

Despite the success of LSTMs, they have some difficulties dealing with very long sequences. The information stored in the cell state can be overwritten along the parsing of sequence, and when the steps are very long ($> 20 - 25$) the probability that they can be lost, during the process, is high. Moreover, the state of each cell is sent only to the following time-step and there isn't much control on how to use multiple previous steps together for the current prediction. To overcome these weaknesses, recently, the Memory-Augmented Neural Networks (MANNs), like Differential Neural Computers (DNC), have been proposed.

MANNs actively use the information contained in an external memory in the inference process. The external memory can be fed along the input [70] and used through special attention mechanisms or included inside the network's architecture with its learnable parameters. DNC, shown in Figure 2.6, belong to the last category: it is a machine learning model that consists of a neural network that learns to read from and write to an external memory [32]. It is the successor of Neural Turing Machine [31], improving it with a more powerful access mechanism to the memory and a clever way to manage it.

The neural network, called *controller* from now on, in principle can be of any type, but the most used one is the LSTM. In fact, the structure of sequential data and

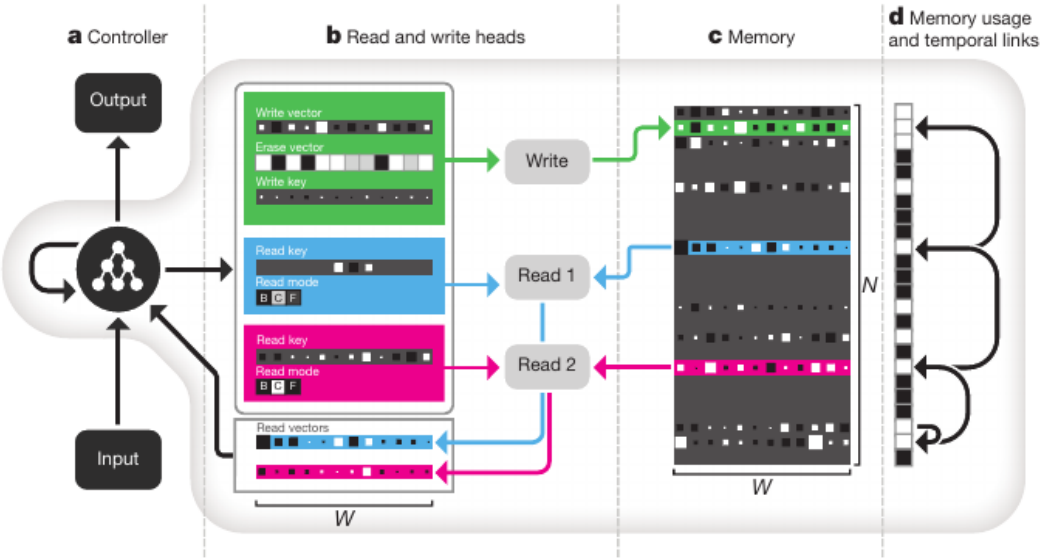

Figure 2.6. DNC architecture, as shown in [32]. The controller output is used by the memory to calculate all the parameters needed by the heads (e.g. erase vector, write key, read key, etc). The content retrieved by the Read Heads is stored in the vectors that are returned to the controller for outputting the final prediction. The last column shows the usage of the memory and the strong links generated by the temporal matrix.

the capacity of LSTMs to produce and to return a state for each element in the sequence, allow the system to use all the advantages of a memory network.

The controller learns to write and to read a matrix $M \in \mathbb{R}^{N \times M}$ the *memory*, through two basic units called *Write Head* and *Read Head*. The symbols $\mathbb{R}^{N \times M}$ denotes the set of real-valued vectors with dimension $N \times M$. Each unit takes an input and elaborates the operations that must be performed on the memory.
In the case of LSTM, the input of each head consists of the state produced by the controller for each element in the sequence.
In order to decide how to perform the operations, each unit first calculates, from the current step output, a list of vectors and keys called *interface vector* ξ and then uses it to produce the so-called *weightings*.
The elements of ξ are:

- $k_t^{r,i} \in \mathbb{R}^W$ and $k_t^{w,i} \in \mathbb{R}^W$, respectively the read key for the Read Head i and the write key for the Write Head i, at the time-step t. They are the encoding of the current state's content, and they will be the quantities that are compared against the memory content in the content addressing mechanism;
- $\beta_t^{r,i} \in [1, \infty]$ and $\beta_t^{w,i} \in [1, \infty]$, the strength of the previous keys that control the importance of such keys;
- $e_t \in [0, 1]^W$, the erase vector that checks how much of the content of a memory cell should be cleaned during the current time-step, before to write in;
- $v_t \in \mathbb{R}^W$, the write vectors weigh the quantity of new information that must

be written in each cell;

- $g_t^a \in [0,1]$, that governs the write mechanism, choosing how much of the information should be written in already written (old) cells or in a new one;
- $g_t^w \in [0,1]$, the write gate, tells whether or not to write in a given cell;
- $f_t^i \in [0,1]$, the free gate for the Write Head i that tells if the given cell can be cleaned or not;
- π_t^i, the read mode that,as it will be shown, balances the different read mechanisms.

The elements in the interface vector are computed starting from the controller state. They are obtained:

- applying the sigmoid function to the input for the erase vector e_t, the free gates f_t^i, the allocation gate g_t^a and the write gate g_t^w;
- as a product of linear layers for the write keys $k_t^{w,i}$, the read keys $k_t^{r,i}$ and the write vector v_t;
- obtained through a softmax layer for the read modes π_t^i is ;
- as a product of oneplus function over the input for the write strength β_t^w and the read strength $\beta_t^{r,i}$.

All these elements will be further described, when used, in the next sections.

The weightings

The weightings are «distributions over the rows of memory and represent the degree to which each location is involved in the operation» [32]. They are the result of the combination of three different access mechanisms, in the forms of differentiable attention: content lookup, the temporal addressing and the allocation mechanism.

The **content lookup** is used by both the Write Heads and Read Heads. It compares a key, representing the input, to the content of the memory and, according to a similarity score, generates the *content weightings* that focus the attention on the memory cells more similar to the key. The chosen similarity score is the cosine similarity, and it is computed, between two vectors $\mathbf{u}$ and $\mathbf{v}$, by the formula:

$$D(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}||\mathbf{v}|}. \tag{2.13}$$

Therefore the content weights are:

$$C(M, \mathbf{k}, \beta) = \frac{\exp\{D(\mathbf{k}, M[i,\cdot])\beta\}}{\sum_j \exp\{D(\mathbf{k}, M[j,\cdot])\beta\}} \tag{2.14}$$

where $M[i,\cdot]$ denotes all cells of the i-th memory row, $\exp\{x\}$ stands for the quantity e^x, $\mathbf{k} \in \mathbb{R}^W$ is the lookup key, $\beta \in [1,\infty)$ is a scalar representing key strength, and D is the cosine similarity. Both the key and β are extracted from the interface vector.

The **allocation mechanism** allows the controller to free and allocate memory when it is needed. It is used by the Write Heads to choose where to write the data.
The allocation weightings are built from the combination of three vectors: the usage vector, the read writings of the previous time step and the write weightings of the previous time-step.
The usage vector fixes the usage and utility of each location, and it is computed by the formula:

$$u_t = (u_{t-1} + w^w_{t-1} - u_{t-1} \circ w^w_{t-1}) \circ \psi_t \tag{2.15}$$

where, the write weightings w^w_{t-1} and the usage vector u_{t-1} refer to the previous time-step, $\circ$ denotes element-wise multiplication, while ψ_t is the retention vector. The latter is given by:

$$\psi_t = \prod_{i=1}^{R}(1 - f^i_t w^{r,i}_{t-1}) \tag{2.16}$$

and tells how much each location cannot be freed. Intuitively, a location is free if the free gate f^i_t of Read Head i is 1 (or close to) and the location has been just read, and hence already used by the network. Therefore, the usage increases for the cells just written and is decreased by the retention vector for some read cells.

Lastly, given the usage vector, it is built a sorted list $\phi \in \mathbb{Z}^N_t$ of the memory locations ready to be used for writing operations, based on the usage vector. The list is used in the final formula of allocation weighting:

$$a_t[\phi_t[j]] = (1 - u_t[\phi_t[j]]) \prod_{i=1}^{j-1} u_t[\phi_t[i]] \tag{2.17}$$

Therefore, if the usage of a cell is 1 then the allocation weight of that cell is 0 and the network is pushed to write on different cells. In the other cases the allocation mechanism also considers the usage of the previous, less used, cells to assign a right weight to it. The higher is the usage of these cells, higher will be the allocation of the current cell.

The last addressing mechanism is the **temporal** one given by the temporal link matrix $L_t \in [0,1]^{N\times N}$. It keeps track of consecutively written memory locations. Each cell $L_t[i,j]$ represents the degree to which each location i was written immediately after the location j. To calculate such a degree, it is computed the *precedence weighting* as:

$$\begin{aligned} p_o &= 0 \\ p_t &= (1 - \sum_i w^w_t[i])p_{t-1} + w^w_t \end{aligned} \tag{2.18}$$

where w^w_t is the write weighting shown below in equation 2.21.
The temporal link matrix is updated removing the old link to and from a location and putting inside the new ones. The logic is implemented as:

$$
\begin{aligned}
&L_0[i,j] = 0 \qquad \forall i,j \\
&L_t[i,i] = 0 \qquad \forall i \\
&L_t[i,j] = (1 - w_t^w[i] - w_t^w[j])L_{t-1} + w_t^w[i]p_{t-1}[j]
\end{aligned}
$$

The temporal link matrix can be used in two-ways: iterating the elements from the oldest to the newest or in the reverse order. Therefore, two types of weighting are produced: the backward weighting b_t^i and the forward weighting f_t^i.

$$f_t^i = L_t w_{t-1}^{r,i} \tag{2.19}$$

$$b_t^i = L_t^T w_{t-1}^{r,i} \tag{2.20}$$

The previous described addressing mechanisms are used to calculate the *write weighting* and *read weighting* that determine the write and read operations.

The **write weighting** (eq. 2.21) chooses if and where to write the current information.

$$w_t^w = g_t^w[g_t^a a_t + (1 - g_t^a)c_t^{w,i}] \tag{2.21}$$

It uses two gates extracted from the interface vector: $g_t^w \in [0,1]$ is the write gate and regulates how much data write into the memory; $g_t^a \in [0,1]$ is the allocation gate and fixes how much of the data must be written in new locations and how much must be written inside cells with similar content. Finally, c_t^w is the write content weighting calculated from the equation 2.14. The meaning of the equation is that if the write gate is 0, then no information are written in memory, otherwise the allocation gate controls how much data store in cells with similar content and how much data have to be stored in new cells.

The **read weighting** (eq. 2.22) is a combination between content addressing and temporal addressing, governed by the *read mode* vector.

$$w_t^{r,i} = \pi_t^i[1]b_t^i + \pi_t^i[2]f_t^i + \pi_t^i[3]c_t^{r,i} \tag{2.22}$$

For example, if the first element of read modes $\pi_t^i[i]$ dominates the other, then the readings are mainly based on the temporal weighting in reverse order.

The use of memory

As described so far the controller can write and read the memory. The last question to be answered is how the network can use such a mechanism to improve its performance.
The key elements that allow the use of information in memory are the *read vectors*. They are a weighted average of the contents of the locations in memory using, as

weights, the read weighting of the previous section, and represent the read of the controller at each time. Each Read Head returns one read vector as:

$$r_t^i = M_t^T w_t^{r,i} \tag{2.23}$$

Each vector can be appended to the input of the next time-step, to increase the information included in the next input, or concatenated to the controller output as input of a layer that will take the final decision about the current input, or both of them.

The output of a write operation is the update of the memory matrix containing the new information and, eventually, removing the old ones. The memory is modified using the following formula:

$$M_t = M_{t-1} \circ (E - w_t^w e_t^T) + w_t^w v_t^T \tag{2.24}$$

where $e_t \in [0,1]^W$ is the erase vector and $v_t \in \mathbb{R}^W$ is the write vector extracted from the interface vector.

Thanks to the read vectors, the DNC overcomes the problems of LSTM. Because it stores the states of LSTM in the memory, it can use multiple states for the current prediction and can use actively also very old states. The use of weightings allows, additionally, to track how it uses them, getting some insights about the decision process.

2.2.4 Deep Q-Learning

As mentioned in the section 2.1.3, a reinforcement learning agent learns a policy that tells, for each state, which is the action that maximize the future rewards.

Therefore, in order to learn the policy, it has to learn the values of each action-state pair, called *Q-values*, that map the pair to the expected reward.

$$Q(s, a|\pi) = r(s, a) + \sum_{i=1}^{n} \gamma^i \max_a Q(s^i, a|\pi) \tag{2.25}$$

For each pair, its Q-value is the sum of the reward obtained selecting the action a plus the discounted sum of the future expected rewards obtained following the current policy π. The γ parameter controls how much important are the future rewards: for $\gamma = 0$ then only the immediate reward counts, for $\gamma = 1$ the rewards obtained from now on have all the same importance. Clearly the process to find the optimal policy alternates between the improvements of estimation of Q-values for the current policy and the improvement of the policy itself.
The process to learn the values is called *Q-Learning* and its basic rule to update the estimation is:

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t)) \tag{2.26}$$

where $Q_t(s_t, a_t)$ is the old estimation, α is the learning rate and $Q_{t+1}(s_t, a_t)$ is the new estimation. Usually, these values are stored in a table, and then inspected by the policy to choose the most valuable action. The table approach has several limitations: the memory needed, to store the table, grown with the complexity of the action space of the problem, becoming unfeasible very soon; it cannot generalize to unobserved states, therefore it needs extensive exploration during the training process to ensure that all the possible states are included in the table.

The neural networks can be used also in these settings, mitigating these limitations. They can provide an approximation of Q-values when exploring all the states is infeasible or the required memory is too large. In these cases can be used two neural networks to approximate both the policy and the estimation of the expected reward from a given state. In the first case the network is called *policy network*, while the second one is the *value network*. Note that now the policy $\pi(s, \theta)$ depends also on the actual parameters of neural network θ.

In the aforementioned context the learning is called Deep Q-Learning, underlining the fact that the Q-values are learned through a deep neural network. Clearly the update equation is fit inside the loss function of the network that is set to:

$$E_n = (r + \gamma \max_{a'} Q_t(s', a') - Q(s, a))^2 \tag{2.27}$$

Finally, in the context of sequential data, the policy network is usually fed with a sequence of observations, like a series of frames for video processing, and it must return the best action, given the learned policy and the observations provided.

2.3 Explainable Artificial Intelligence

Although the great success of AI, there are still some domains where AI systems have limited application due to ethical concerns and a lack of trust of the users [53]. The lack of trust is mainly caused by the fact that the most recent tools of artificial intelligence, like deep neural networks, are seen as *black boxes*. Typically, you feed the data into the network and take the output from it, without the possibility to see how the decision process of the network works, neither to get hints of explanation about the results, making these models hardly interpretable. The absence of interpretability makes difficult their adoption in domains, like health care, where the availability of explanations, that allow the human experts to validate the network predictions, is a prerequisite.

Despite the clearness of the problem, there isn't a standard definition for *interpretability* in Artificial Intelligence yet. In fact, the term cannot be linked to a single concept, but reflects several distinct ideas [45]. Some authors [11, 28] propose a distinction between *interpretability*, defining a system as interpretable if its operations can be understood by a human, for example through introspection, and *explainability* that explains why a decision is a good one, but not necessarily explaining how it was made.

Note that, while in principle we wish to unfold the complete steps of reasoning and track accurately the way in which an AI takes a decision, you have to consider that not even the humans can fully explain their decisions: in fact they can give you some useful information or motivation about the decision, but they cannot explain all the steps that their brain does to make the decision. Therefore, the full interpretability of a complex model, like the neural network in brain or artificial intelligence, seems to be more and more hard to obtain than the explanations. Furthermore, especially for non-technical users, it is more useful to check an explanation, that summarizes the information used by the network, instead to study all the (possibly long) steps done by the network to produce the result.

The field of eXplainable Artificial Intelligence (XAI) collects all of these considerations and tries to get a solution for them. The easiest and oldest method, to get explanations, is to use statistical measures, like the study of marginal effect of one feature on the prediction using Partial Dependence Plots (PDP) [26] or Individual Conditional Expectation(ICE) [29]. Despite their simplicity, they have limited capacity to deal with domains containing a large number of features or highly variable features, as in text and image problems, the most common problems solved by neural networks. More recent works, instead, focus the attention on the properties of the networks or try to approximate their behavior, getting better results for more complex problems. They are described in the next chapter that gives an overview of the major state-of-the-art approaches.

One interesting question regards the features or shape that the explanation should have. Miller [53] extracts, from social sciences, some features of the most useful human explanations. They are usually:

- *contrastive*, meaning that the humans usually prefer to compare two events to understand why one happened instead of the other;
- *selected*, meaning that they are only a small subset of all possible reasons behind the decision, and that the chosen subset is biased by the preference of each person;
- *social*, because they depend on the belief of both explainer and explainee.

Additionally, the humans tend to ignore the probabilities associated with the explanations, giving them poor importance. While these findings must be taken in consideration to increase the trust of the users, they can be useless for debugging and understanding of deep neural networks, another of the goals of XAI.
In these latter contexts, there are important other aspects, for example the representativeness of the model, the formal guarantees of the explanations quality, their direct connection with the decision process, the probabilities and so on.
An approach that satisfies all the features at the same time, for both the cases, is a utopia, but it is also very hard to fulfill only some of them.

2.3.1 The problem of evaluation

The evaluation of explanations is an open problem of Explainable Artificial Intelligence. The difficulties arise from the absence of ground truth for testing the methods, and the fact that the proposed approaches cover several aspects that cannot be unified for research purpose. In fact, they depend on the particular environment, context and system that will be used.
For example, usability studies can be appropriate for systems that should help non-technical users, but they are inadequate to assert the validity and fidelity of the returned explanations. Consequently, there isn't a universal metric to assess the validity of an explanation method, but there are various methods, each specific to a subclass of problems.

Ancona et al. [5] propose an experiment where the model is tested on many versions of input, where a part of it is occluded. From the obtained collection, you can extract which parts cause a model to change its prediction and you can extract a "*surrogate ground truth*" from which you can evaluate your approach. Clearly the method is computationally inefficient, and you have to be careful in the selection of experiments from which you will extract the ground truth.

For approaches that use the activities of inner components for explanations, for example tracking the activation functions of the neurons, a possible solution is to test their power in the so-called *transfer learning*. If a layer it is specialized in the detection of a particular feature, then it should work, in the same way, also if moved in an architecture where the following layers work in a different way or the objective of the neural network is changed.

An apposite and popular approach, that focus its attention only on the accuracy side, is to remove the supposedly important features from the input and check if the model performance degrades. The drawback is that it is unclear if the degradation is due to the importance of the feature or due to the violation of the assumption that training and test data come from the same distribution [37]. The alternative is the progressive replacement of the selected features with a constant value (e.g. the mean value of that feature)[61, 37]. If the selection starts from the highest-ranked feature the associated metric is called *MoRF* (Most Relevant First), while in the opposite case is called *LeRF* (Least Relevant First) [6]. MoRF and LeRF are effective when there are a few key features inside the example, while encounter some difficulties when dealing with overabundant data, for example in the case of multiple occurrences of similar objects in an image.

Finally, consider the case of human evaluation. First of all, note that an explanation that seems unreasonable could be linked both to a failure of the system to process the input and to a failure of the explanation system [28]. Conversely, a more human-like explanation can be produced purposely by a system, ignoring at all the decision-making system of the black box. Therefore, evaluation methods based only on visual inspection or human evaluation are intrinsically unfaithful [5] because they are biased on the superficial appearance of the explanation instead of their accuracy [23].

In addition, it has been proved that the trust and decision process of a human are profoundly different between the case study and the real case scenario, and they are influenced by personal belief and attitude hard to isolate [15]. Hence, the human evaluation is a poor guide also for systems that will be employed along with a human operator.

The problem becomes even harder when you have to decide which method is the best for your domain. There are only a few attempts to compare different approaches from a theoretical perspective [5] and none of them is exhaustive [37]. The architectures considered by the methods are often incompatible and there isn't a common benchmark yet where you can compare the approaches. Furthermore, when the methods appear similar and comparable, like Input Gradient and Integrated Gradient described in the next chapter, could be the case that they provide answers to different questions [5].

Chapter 3

State of The Art Approaches

In this chapter there will be reviewed some of the most recent approaches to address the problem of providing explanations that summarize the behavior of a black box model. The chapter doesn't cover approaches like neural caption generation or external justification models, that aim to generate explanations along the prediction using another black box model, because they remain obscure and out of scope of the thesis.

3.1 Global Approximators

The first works concerning explainability are mainly based on the construction of a function f that approximates globally the black box models g [33].
The most used functions for f are the decision trees, the linear models and the set of rules, because they are easily interpretable. While for the linear models the interpretability comes from their weights, directly linked to the decision process, the decision trees, instead, exploit their graph structure. They are composed of internal nodes representing features and edges that represent tests on the features. Each branch represents a possible partial assignment of values to the features and its output (Figure 3.1), and it can be interpreted as an $if - then$ rule in the form of conjunctions of conditions on features.

The approaches differ from each other for the way in which they build the interpretable models and the size of them.
An example of the linear models is the method proposed by Lou et al. [46] that exploits Generalized Additive Models (GAMs) to interpret regression and Tree Ensembles. Their method returns the importance of the contribution of a single feature using "shape functions", single-feature models that capture the connection between a feature and the output.

For decision trees, instead, *Treepan* [20] and its new versions [42] query the black

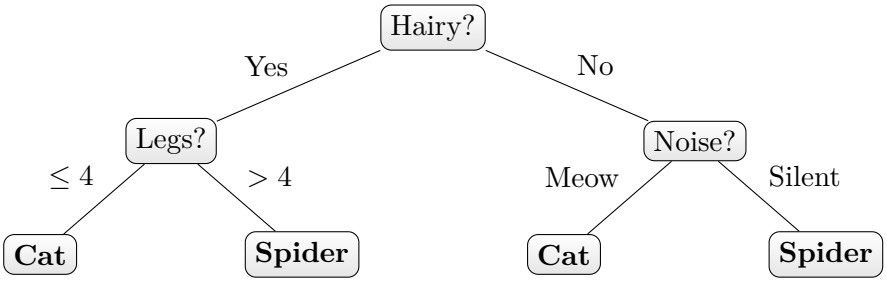

Figure 3.1. Example of decision tree. The bold nodes are the predictions, while the inner nodes describe the features, and the edge labels are the tests on them.

box multiple times, collecting the answers and building the decision tree f. They select, at each time, an instance from the dataset, to be fed into the network, that helps to split adequately the nodes in the tree and, simultaneously, maximize the gain ratio and the fidelity respect the original model. Decision trees and rules can be also extracted using genetic algorithms [44, 40] or constraint programming [27].

To reduce the dimensions of decision trees, and consequently to increase the readability and explainability, can be first computed "prototypes" of the training data, obtained using statistical analysis, knowledge of the problem or genetic algorithms [44, 72], and then use the prototypes as a guide for the building the decision tree. An alternative is to prune the obtained tree, for example, deleting the branches that lower the fidelity [13].

These types of approaches are mainly used to explain Tree Ensembles methods, like random forest, where they can also exploit the common structure with them. Indeed, because they are a conglomerate of trees, an idea can be, for example, to shrink the forest, summarizing its properties through clustering in a few, more understandable, decision trees [17].

Regarding the evaluation of these approaches, the most used metric is the gap accuracy, that measures the difference between the accuracy of f and g, to evaluate the representative power of f.

The main drawback is that the approximation must balance the comprehensibility of their process and the representativeness of the black box, and the compromise often isn't enough good for critical problems. Additionally, often they work only for tabular data leaving out many of important domains. Consequently, the successive works tried to shift the approximation from a global perspective to a local one [57], meaning that the explanation model is required to represent the model only for the current prediction, relaxing the need for balance.

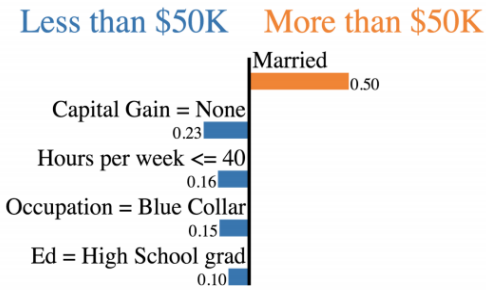

Figure 3.2. A LIME application to explain a prediction on tabular classification (adult dataset). The task goal is to predict if a person makes $\geq$ \$50k annually based on its personal data. Orange bars indicate the features that shift the prediction towards "more than \$50", while the blue ones towards "less than \$50k"

3.2 Local Surrogate Models

Local surrogate models try to train another, more interpretable, model that approximates the black box only on the current prediction. The approaches considered in this section, called *additive features attribution methods* [47], typically assign at each input feature a score, that represents its importance for the inference phase. The most used approaches are LIME [57] and SHAP [47].

LIME probes the model perturbing the current input, collects the modified inputs and train an auxiliary classifier, like regression model, on the new set of examples, called *synthetic data*. Finally, it uses the trained model to explain the current instance, ranking each feature with the weights of the trained model. Clearly the new model has only local validity, so it isn't required that it represents the whole black box, but only that can approximate its behavior on the current instance.

SHAP [47], in its kernel version, is a generalization of LIME changing the way in which the synthetic data and the weights are computed. They used a game framework to estimate the contribution of each feature, approximating the Shapey Values. Each feature, or set of features (coalition), plays a game against the others to win the gain, represented as the difference between the average prediction and the current prediction. One of the differences between LIME and SHAP is the way in which the examples are chosen to train the classifier: LIME chooses the synthetic examples closer to the current instance, while SHAP chooses them according to Shapey value estimation, privileging both the smallest and the largest coalitions. In this way the classifier is trained with examples where each feature is considered both in isolation and when only that feature is missing, returning a more accurate estimation.

The advantages of these approaches are that they are model-agnostic, meaning that they are, in principle, applicable to any type of neural architecture, and the returned scores are easily understandable for both expert and non-expert users. The main problem is that the process of finding useful neighbors for the current instance, from

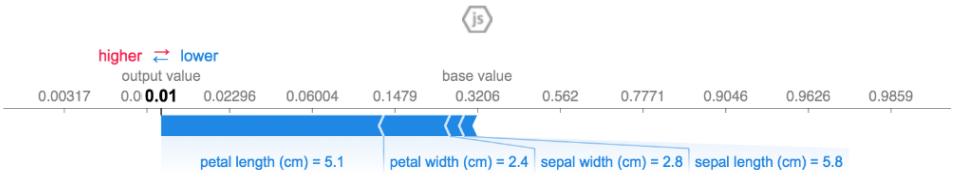

Figure 3.3. SHAP application for SVM prediction on IRIS dataset. The goal is to predict the species of flowers based on its features. The image shows the most important features for the current prediction, that shift the output from the average output.

which you can collect the dataset to train the interpretable model, is problematic and depends on the parameters of used kernels. For example, the Gaussian distribution from which the examples are sampled is inadequate for problems where there is a strong correlation between features inside the same instance. Moreover, they require plenty of time to create the samples and train the classifier, especially if you are dealing with problems with lots of features. Finally, Alvarez-Melis and Jaakkola [4] find that these types of approaches are easy to be fooled: two examples very close can produce different explanations making them unstable and difficult to trust.

3.3 Set of Rules

A set of rules describes the logic behind a black box providing rules and constraints that bind features and output.

Craven et al. [19] propose to start from an empty set of rules and then analyze iteratively the training data: for each sample, if its features aren't linked to the output with a rule, then a conjunctive m-of-n rule is created starting from that example. An opposite method, that focus the attention on the network instead of the dataset, is developed by Augasta et al. [7] and proposes to prune input neurons and identify and store the data range needed by the network to return a specific class. At the end of the process, all data ranges are collected and, from then, the rules are extracted for each class in the dataset. In a similar way to the decision trees, the rules can be extracted also using genetic algorithms [41] or prototypes [56].

The rule extraction methods are mainly used to explain Support Vector Machines (SVM) [41] and Tree Ensembles [21] until recently, where the set of rules has emerged again thanks to the proposal of the authors of LIME. Their approach [58] uses the local approximation principles to build a set of rules, called *anchors*, applicable to any neural networks.
They are computed using reinforcement learning combined with graph search to reduce the required time to approximate the output. A rule returned by this approach anchors the prediction if changes in any of the other features don't change the output. The novelty is that the rules have a precise coverage, and they can potentially be used to explain other instances of the dataset, reducing the total

```
EQUATION RULE
IF A*X1 + B*X2 + C*X1*X2 < G
THEN class 1

INTERVAL RULE:
IF X1 in [a,b] AND X2 in [d,e]
THEN class 1
```

Figure 3.4. Example of rule extracted from data ranges.

```
IF x1 == valueA
AND x2 == valueB
THEN PREDICT ClassA
WITH PRECISION 90%
AND COVERAGE 20%
```

Figure 3.5. Example of rule extracted by the anchors algorithm [58].

amount of time required to generate explanations for the whole dataset.
An example of the explanation returned by the algorithm is shown on Figure 3.5

The weakness of anchors, and in general of the set of rules, is that the effort to construct a particular rule is infeasible and you have to approximate its calculation, for example using the reinforcement learning and perturbation techniques. Their use means that the goodness of explanations depends on configurable parameters combined from the different components, that depends also on the problem considered. Therefore, while the approaches can be often model agnostic, their applicability in different domains is unclear. Additionally, you have poor control on the complexity of the rules, so you can get rules that are too general or too specific, for the given example, and on the nature of the rules themselves, therefore they could potentially be contradictory. On the other hand, its output is easy to interpret and the coverage of multiple objects (potentially unseen) is very appealing.

3.4 Backpropagation-based methods

Backpropagation-based methods integrate the backpropagation algorithm with elements useful to the explanations or extract information from it through the gradient. The main advantage is that they are fast because they don't require neither to perturb the input nor to fit another model.

A simple example of this category is the work of Skrikumar et al. [65], that takes the partial derivatives of the output with respect to the input and multiplies them with the input itself. The Integrated Gradients approach [71], instead, consider the average gradient, calculated shifting gradually the input from its true values to a baseline chosen by the user.

More elements are introduced, in backpropagation, by Layer Relevance Propagation (LRP) [9] and DeepLIFT [64], that propose to use purposely designed propagation rules for pushing the output layer by layer, till the input, assigning higher relevance scores to the neurons that contribute the most. They consider, in the backward phase, also some *prototypes* of the chosen output as reference values for the activation functions, to adjust the relevance of neurons.

The output usually is an heatmap that highlights which part of input triggered a layer or a neuron. For this reason, like the saliency maps methods, that we will see in the next section together with other backpropagation methods, it is hard to link the information retrieved by these approaches to explanations that summarize the reason behind the decision process, in a concise and understandable way for the human operators.

3.5 Saliency Maps

Saliency Maps approaches try to build visual explanations about the decision process producing a map that highlights the subset of original data mainly responsible for the prediction. They are very similar to the approaches of the previous section, but they often have also alternative methods that don't use the gradients or information from backpropagation.

Zeiler and Fergus, for example, propose to use a Deconvnet [75, 68] to visualize the features learned by a Convolutional Neural Network (CNN). A deconvnet is a model that uses the same components and weights of the CNN, but in reverse order. Therefore, using it, you can map the activation of each layer to the input space, creating a saliency map that shows for each layer the parts of the input that most excited the given layer's neurons.
A similar result can be obtained through the partial occlusion of the input and computing the gradient [66] or checking iteratively how the prediction changes [49]. Other approaches that produce maps are the Class Activation Mapping(CAM) [76] and its gradient version Gradient-weighted Class Activation Mapping(Grad-CAM) [62], that use the convolutional layers before the softmax layer as unsupervised object detectors [14] pointing out the regions of an image responsible for a prediction.

These methods are very useful to check that, the neural network, focus its attention on right regions of input, but become more challenging extracting information about the motivation behind the choice of a specific class. For example, in Figure 3.6, you can see that the regions highlighted for all the classes are similar and it is difficult to extract the difference between each class. In fact, Adebaio et al. [2] have shown that a considerable number of these methods suffer from the problem to be invariant respect to the labels.

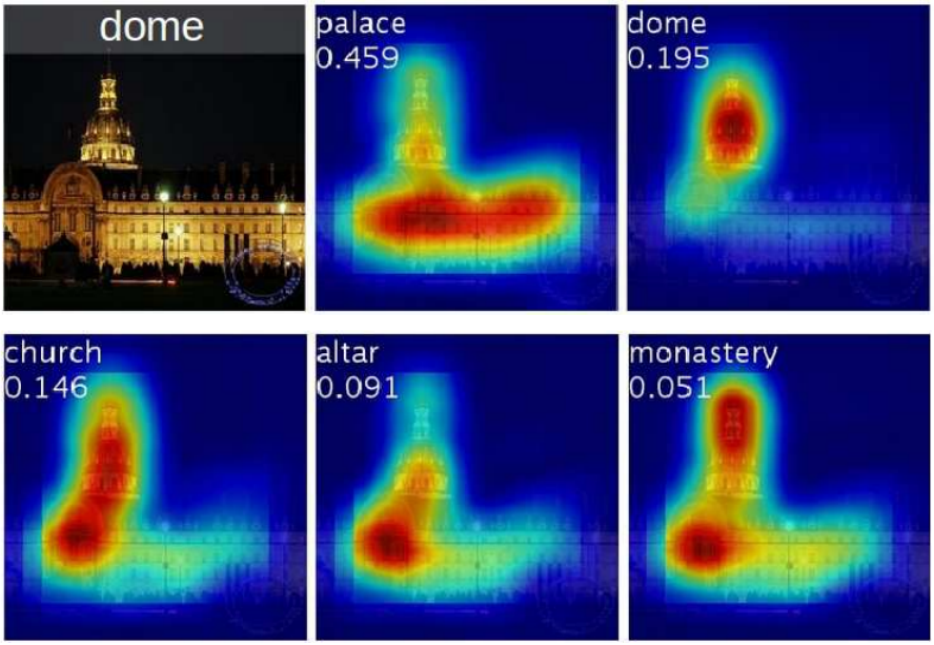

Figure 3.6. Example of visualization technique using Class Activation Mapping(CAM). The figure shows the predicted class and its score. The highlighted regions vary across the classes, but it is very challenging, for a non-expert user, to understand the difference between them.

3.6 Counterfactual Explanations

The last category of approaches regards the counterfactual-based methods [73], that select instances of a dataset to explain the behavior of the model.
The explanations returned put emphasis on structure like "If $feature\#1$ and $feature\#2$ weren't in the instance, then the class C wouldn't have been chosen".
The objective of these methods is to find relations of type cause-effect, between features and prediction, and in particular to find the smallest change in the current instance that changes also the prediction. It can be useful for image dataset or text dataset where each example can contain enough context, while it has some difficulties with tabular data with many features.

The methods to extract counterfactuals are mainly based on the use of meta-rules learned by one of the previous approaches, prototypes, or exploiting the structure of some models. For example, Sokol et al. [67] use the decision trees, where the leaf are the examples, to find a counterfactual as the leaf, linked to a different prediction, that minimize a chosen distance.

The production of counterfactual in XAI is relatively new and under development, therefore there are still many problems to be solved: each extracted counterfactual is isolated and you can get two counterfactuals in contradiction for the same instance; the number of counterfactual, that can be generated, is very large or infinite, and it is difficult to identify which of them are the best candidate to explain the

prediction [16]. Despite that, an approach based on counterfactual appears to be very attractive because it is faster compared to the local approximators and needs only the access to the prediction function of the model and a dataset, potentially different from the training data. These features make the approach attractive for companies that don't want to disclose their models and data, or for neural networks that work in online settings.

3.7 Rigorous Global Explanations

In order to avoid the problem of comparison described in the previous chapter, some recent works try to build explanations that provide some sort of formal guarantees of quality by themselves.

A first term to measure the quality of an explanation can be check how much close it is from a minimal subset of explanations. A minimal subset of explanations is called *prime implicant* if it is sufficient to explain the prediction and any subset of it cannot do the same. The concept is introduced by Shin et al. [63] and used in Bayesian Network Classifier to extract compact explanations.
Ignatiev et al. [39] try to apply the same concept to a broader range of architectures, using abductive reasoning. Their approach can be applied to any machine learning models that can be expressed as a set of constraints in first-order-logic. It consists in the transformation of both model and explanations in formulas, and then, from the model formula, it is computed the minimum-cardinality explanations by a logic solver. At the end the minimum-cardinality explanations obtained are compared to the actual provided explanations to check how far are these from the optimal solution.

Unfortunately, currently, the encoding of a neural network in a logic formalism is an open and recently emerged problem, and there are only few works available, focused mainly on multi-layer perceptron. This means that, while the proof can be correct, it depends by the encoding of the network that is error prone. Moreover, its applicability is limited to few basic models and its scalability has to be proven [39].

Chapter 4

Methodology

The chapter describes the tasks to be solved and explained by the system and methodology used, highlighting the changes of the original DNC implementation and introducing the module responsible for the production of explanations.

4.1 Problems of interest

The system is tested on two problems that adopt sequential data: the Story Cloze Test (SCT) and the T-maze problem.

The first is a natural language understanding task, and the sequence will contain words, each represented in its own way. Based on the meaning of the words, the model has to choose the more plausible ending, represented again by a sequence.
In the second task, instead, an agent moves in an environment, and the system has to choose the action to be performed by the agent, given a series of information collected in the previous time-steps. Here the sequence is the collection of information, and the output is the selected action.

In literature, in order to handle both the problems, it is usually used an LSTM network, because the tasks involve a temporal correlation between the elements in the sequence. In the later chapters it will be shown that also a Memory-Augmented Neural Network can deal with them, adding some desirable features to the outcome.

4.1.1 The Story Cloze Test

The Story Cloze Test [54, 55] is a commonsense reasoning framework for evaluating natural language understanding. In the task, an agent has to choose between two possible endings for a set of stories composed by four premises (sentences) each. In order to be solved, it requires that the system can successfully link the various

Story	Right Ending	Wrong Ending
My friends all love to go to the club to dance. They think it's a lot of fun and always invite. I finally decided to tag along last Saturday. I danced terribly and broke a friend's toe.	The next weekend, I was asked to please stay home.	My friends decided to keep inviting me out as I am so much fun.

Figure 4.1. Example of a Cloze Story Test. The story consists of four premises separated by full stops. Two possible endings are given, one of which is the right choice.

levels of semantics to commonsense knowledge.

Given a story, the system objective is not only to predict the right ending, but also to explain its decision, choosing which one of the premises is the most informative.

The chosen dataset comes from the competition 2016[1]. Each example is self-contained, meaning that it has all the information needed to return the right ending, and it is overabundant, meaning that the correct answer can be inferred from multiple pieces of the story. The task is difficult to solve, using only statistical learning, because the examples in the whole dataset don't share any particular feature and consequently they are highly unrelated.

Srinivasan et al. [69] note that, among the premises, one is typically more informative than the others, pointing with its content to the correct ending. Conversely, some examples contain one premise that purposely could mislead the model to choose the wrong ending. It could be interesting to include this information inside the explanations. For instance, in the story reported in table 4.1, the most informative premise might be the last one, while the second premise could mislead the network due to its similar meaning with the wrong ending.

4.1.2 T-maze

T-maze is a non-Markovian discrete control task that evaluates an algorithm's ability to learn to remember observations and correlate events far apart in history.
In the original task [10, 74], an agent has four possible actions (move North, East, South, or West) and must learn to move from the starting position, at the beginning of the corridor, to the T-junction. Here, it must move either North or South to a changing goal position, that depends on the symbol (N or S) that the agent has seen at the starting position. If the agent takes the correct action, it receives a reward of

[1] https://competitions.codalab.org/competitions/15333

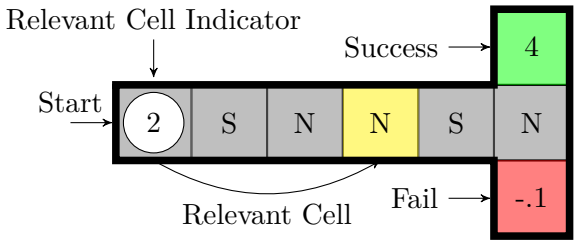

Figure 4.2. Example of a 5-step modified T-maze, where the starting position (in white) indicates the relevant cell (in yellow). This cell contains the symbol N, which indicates that the agent is going to get a positive reward going north in the T-junction, and a negative one going south.

4, otherwise -0.1. In both cases, the episode terminates and a new episode starts with a new goal position randomly set North or South.

The task is modified adding a symbol (S or N) observable by the agent at every step of the corridor, and specifying, in the starting position, the relevant step that contains the reference value to correctly choose the direction at the junction. In this way, the objective of the system is to learn and remember previously seen observations for a long number of steps and justify its decisions by indicating the read symbol and the relevant corridor cell, that corresponds to the ground truth for the explanations.

4.2 Simplified DNC

From the original implementation of the DNC, there are removed all the elements that can hurt the performance in the considered tasks or that can make harder to get valuable explanations.

First of all it is removed the temporal linkage matrix. It, in fact, has a huge impact on the training time, and its usefulness is highly task-dependent. In the STC case, it often reduces the overall network performance – probably due to long sequences with sparsely related information (e.g., sentences with relevant words that are typically separated by several elements like articles, punctuation, adjectives).

Hence, the read weighting, for each head, will be based only on content similarity computed be the equation 2.13:

$$\mathbf{w}_t^{r,in} = C(M_t, \mathbf{k}_t^{r,in}, \beta) = \frac{\exp\{D(\mathbf{k}_t^{r,in}, M[i,\cdot])\beta_t^{r,in}\}}{\sum_j \exp\{D(\mathbf{k}_t^{r,in}, M[j,\cdot])\beta_t^{r,in}\}} \tag{4.1}$$

where in is the index of Read Head.
Notice that the inclusion of temporal linkage matrix can shift the weights focus on different cells, but it doesn't change the shape of them, therefore the method explained in the current and the next section is still applicable with or without it.

Regarding the read vectors, computed from the read weightings using the equation 2.23, they are concatenated to the controller output and sent, as input, to the final linear layer. The input and output of the layer are:

$$y_t = W_c h_t + W_r[r_t^1; ...; r_t^R] \tag{4.2}$$

$$out_t = W_{out} y_t \tag{4.3}$$

where y_t is the inputs at time t, out_t the final output, h_t is the controller output, r_t^i is the read vector of Read Head i and W_r, W_c and W_o are adaptable weight matrices.

As said in the previous chapter, in the original implementation [32], the controller can use content-based addressing to select the cells where to write, decreasing the memory usage, thus (partially) overwriting previously stored information. While the mechanism allows to reduce the required memory-size and to store new information about old facts, it can generate an inconsistency on the explanations.
For example, in data sequences containing closely related information (e.g., multiple facts about the same subject in a story) the same memory-cell could be repeatedly used, due to the content-based addressing. The overwriting process would make hard to both establish which part of the sequence added more information and to provide an explanation of the prediction.
As a result, the content-based addressing during write operations is disabled, and the possibility for the network to overwrite cells is removed, obtaining a more stable behavior. Specifically, considering an $N \times W$ memory matrix M_t, the write weighting vector are transformed from equation 2.21 to:

$$\mathbf{w}_t^w = g_t^w \mathbf{a}_t \tag{4.4}$$

Finally, based on the analysis of Franke et al. [25], there are applied the *bypass dropout* to the output of the LSTM to force an earlier memory usage during training, and a *layer normalization* to the input of memory in order to stabilize the training process.

The bypass dropout limits the impact of the controller to the final output, reducing its connectivity in the training phase.
Let be d_t a vector of the same dimension of controller output, obtained using a Bernoulli sampling - each of its elements is set to 0 with a fixed probability p. Then d_t is multiplied with h_t, a portion of the input of the last layer, «shutting down» randomly some elements of the controller output. The inputs of the final layer become, in the training process:

$$y_t = W_c(h_t \circ d_t) + W_r[r_t^1; ...; r_t^R] \tag{4.5}$$

The bypass dropout has proven very useful for the system: it is a key element to obtain good explanations and to solve some tasks in a stable way. The forcing use of memory, in fact, allows to retrieve the most used element for the prediction and make the whole training process more stable.

The layer normalization [8] is a technique that is know to improve the stability of the recurrent neural networks, centering and scaling the activation at every time step. In the system, it is applied to the controller output, before it is fed to the memory to produce the interface vector. Let be μ_t the mean of a vector x_t, σ_t^2 its variance, and b and g two learnable parameters, and ϵ a small value to avoid the square of 0, then the layer normalization is given by the equation:

$$LN(x_t) = g \circ \frac{x_t - \mu_t}{\sqrt{\sigma_t^2 + \epsilon}} + b \tag{4.6}$$

4.3 Explanation Module

The explanation module has the goal to track the memory usage during the prediction phase and to provide explanations based on that.

As said before, the memory contains information extracted from the states of the controller at each step, and the write and read weightings describe memory usage. More in detail, a high write weighting for a cell indicates that the same cell will contain most of the information of the current LSTM state after the write operation, while a high read weight indicates that the cell contains information similar or related to the current state – due to content-based addressing.
In this context, there can be easily collected inference explanations by noticing that the encoding of a prediction should produce similar states to the ones, stored in memory, that are obtained during the relevant parts of the input sequence.

Note again that the intuition depends on the considered tasks and by the simplification that it is described in the previous section, but it is valid also for different types of configuration of the DNC. The only thing that matters is that the memory is used actively during the prediction and that the read weightings are a *sign* of the importance of the elements in the sequence.

The proposed approach (illustrated in Figure 4.3) records the content of a subset of memory-cells at each step, selecting them on the basis of the strength of the weightings. Each cell written during the input sequence and each cell read during inference is collected and linked to the recording time-step and to the input that generated the stored state.

For writing operations, there are tracked only written cells whose weights are above the mean weighting value. Note that tighter thresholds would not be more informative, since most of the times only one or few cells are written and most of the cells have write weightings close to zero.
Conversely, for read operations, while there are considered only the weights above the mean to prevent the useless reading of the whole memory, there can be used several tight thresholds extracting and storing the *top-*N_c read cells, i.e. the N_c cells with highest read weights.

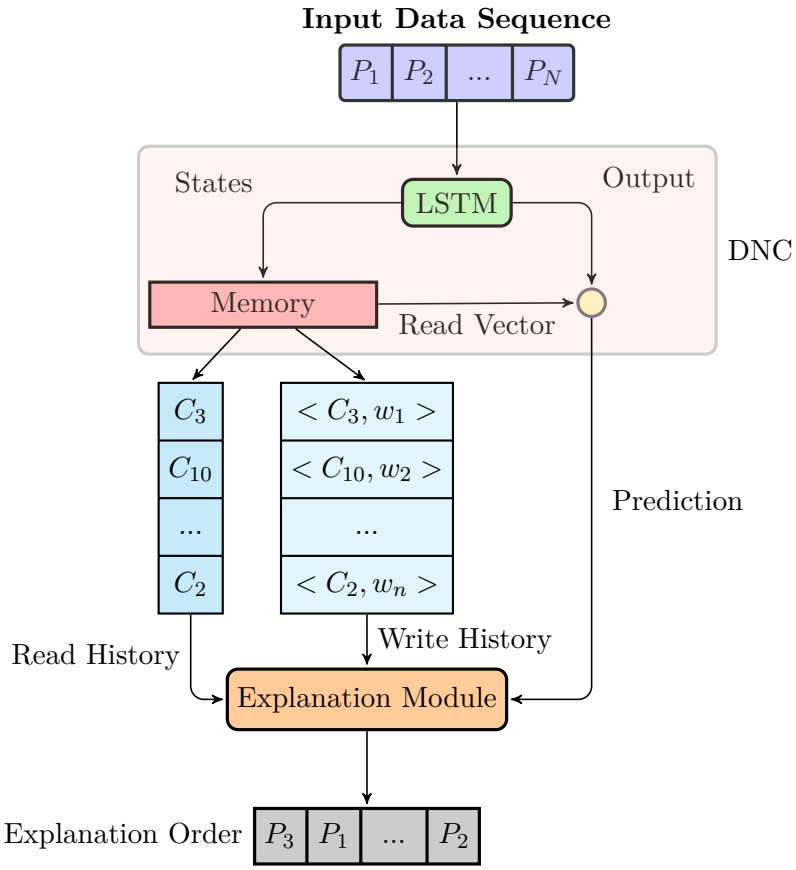

Figure 4.3. Sketch of the system architecture. The blue box represents the sequential input (e.g., from the Story Cloze Test), while the pink box represents the classical DNC architecture, highlighting the read vector produced by the memory. From the memory, the read and write history are fed to the explanation module. Specifically, $< C_i, w_j >$ indicates that word w_j is stored in cell C_i. The output of the explanation module is a vector (at the bottom of the figure), containing an ordered list of explanations. The order is determined by their relevance.

Based on this, it is computed the history of memory usage and it is built a ranking of prediction explanations by means of the frequency of cell readings. Clearly, when using small N_c values, fewer cells are available in the history. Conversely, by setting N_c to the maximum possible value, the whole memory is included in the history.

Since the controller state at any time t depends on (almost) all the sequence until that time-step, the explanation ranking cannot depend on a single sequence element – that does not provide sufficient information about the current prediction. Contrarily, it considers longer chunks of the input sequence and provides explanations on those.

From the explanation ranking, it can be extracted the *best* and *worst* explanation, as the sub-sequence that contains – at prediction time – respectively the most read steps and zero (or minimum) read steps.

The explanation module is particularly well suited for tasks where background knowledge is first fed to the network, and the model is subsequently required to answer specific queries – e.g., question-answering tasks. However, in different domains – such as vision and robotics – the approach could be easily adapted by modifying the addressing mechanism from content-based addressing to temporal linkage.

Chapter 5

Experiments and Results

5.1 Configurations

The system is implemented using Python, Tensorflow 2.0 [1] and Sonnet, an open source library built on top of Tensorflow and designed by Deepmind. Starting from the official DNC implementation provided by Deepmind, it was modified as explained in the previous chapters and adapted to work on tensorlow 2.0.
Regarding the parameters, the DNC controller is configured to use a single LSTM layer composed of 128 units and a probability of 0.2 is fixed for the bypass dropout. For T-maze it is used the RMSProp optimizer with a learning rate of $1e^{-3}$, while for SCT is used the Adam optimizer with a learning rate of $1e^{-4}$; in both the cases the gradient is clipped with a value of 20. For the memory parameters there are used: a memory of size 512, 4 Read Heads and 1 Write Heads for SCT and a size of 50, 1 Read Head and 1 Write Head for T-maze.

5.2 T-maze

5.2.1 Representation

T-maze has a fixed corridor length of $N = 25$ steps, and the relevant cell indicator is computed picking randomly $n \in [0, N]$ and representing it as a real number. The symbol, observed by the agent at each cell, is represented as a discrete value in the set $\{0, 1\}$, while actions are represented as a one-hot vector: [0 , 1] for the *move North* action and [1 , 0] for *move South*. The example in Figure 4.2 would be represented as the vectors:

$$x_i = [\ 2.0\ ,\ 0\ ,\ 1\ ,\ 1\ ,\ 0\ ,\ 1\ ]$$
$$y_i = [\ 0\ ,\ 1\ ]$$

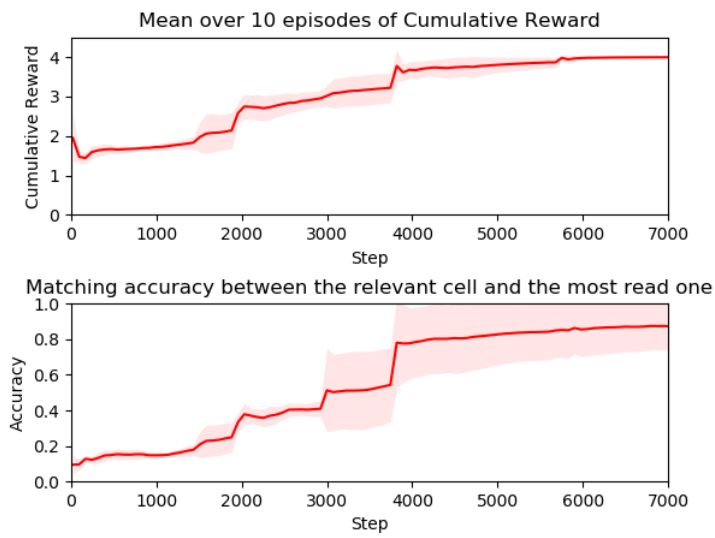

Figure 5.1. The top plot shows the mean and standard deviation of the cumulative reward obtained during training over 10 different runs. The bottom figure, instead, shows the matching accuracy between the relevant corridor step and the 2 most-read memory cells.

5.2.2 Experiments

As mentioned, the modified T-maze task is used as a toy-problem useful to validate the idea. The main questions is whether it is possible to successfully solve the partially observable Markov Decision Process by replacing standard LSTM networks with the simplified DNC architecture, and whether the explanation module works as expected. To this end, first, the overall performance of the model in solving the control task is evaluated, and then the explanations provided by the agent are analyzed. The network is trained running the Deep Recurrent Q-Learning algorithm [34] on 1500 episodes and testing the learned model on 500 episodes with the γ parameter fixed to 1.

To evaluate the goodness of explanations, it is used the knowledge about the problem. In fact, by construction, the only information that the network can exploit are the content of the first cell and the content of the relevant step corridor. Therefore, an explanation should contain at least one of them to be considered acceptable.

5.2.3 Results

Figure 5.1 (top) presents the average cumulative reward of the agent over 10 different training runs. The results show that the model gradually converges to a solution in which the agent can consistently achieve the highest reward. Hence, we can conclude that architecture can successfully replace standard LSTM networks in this domain.

Additionally, Figure 5.3 (bottom) shows the accuracy of the explanation system, obtained by checking whether the information from the relevant corridor step is contained in the two memory cells that are the most-read when the agent reaches the T-junction. There are considered two cells instead of one to also account for the first corridor step, that could be used equally often.
The plot indicates that, most of the times, the agent justifies its decisions with the observation contained in the relevant corridor step, thus confirming the expected behavior for the explanation module.

5.3 Story Cloze Test

5.3.1 Representation

In order to feed to the network the sentences of the Story Cloze Test dataset, it is needed to convert the sentence format from string to real numbers that can be understood by the network.

The first phase is to *tokenize* each sentence extracting, from each of it, all its words. While the process seems simple, it presents some difficulties, for example when dealing with composed word. To accomplish such a goal, it is used a freely available software called Stanford CoreNLP [48], extensively tested by the community.

Once that the words are extracted, each of them is *encoded* in a 300-dimensional vector using Word2Vec [52], a method to construct the word embedding. A word embedding is a representation of the words in a n-dimensional space such that it is able to capture the semantic and syntactic similarity between words, relation between them or the context in which they are used. This means that the representation of two words like King and Queen is very close, despite the fact that the superficial structure (syntax) of the word is different. The similarity is measured by the cosine similarity shown in 2.13 and helps the neural network to improve its performance in many tasks.

A zero-vector of the same dimension is appended to the end of each input to mark the end of the sequence and to request an answer from the network. Moreover, three boolean flags are added to each word, indicating whether the current word is taken from the story, the ending, or the query respectively.

5.3.2 Experiments

The model is trained and tested on the official development and test dataset of the challenge. The former is randomly split into a training and development set containing respectively 90% and 10% of the original dataset.

Regarding the encoding of premises and endings, it is used the same approach

of Mihaylov et al. [51] for the baseline neural model. First of all, the premises are encoded using the DNC cell; then the output encoding is used as the initial state for the encoding of each possible ending producing two ending encoding. The concatenation of them is used as input for a final linear layer that takes the decision, choosing which ending is the correct one.

Task Baseline Comparison

As a first test, it is evaluated the performance of the architecture on the ending prediction task. The goal of the test is to show that the proposed architecture does not under-generalize with respect to existing baselines, such as that of Mihaylov et al. [51]. Specifically, while their work is not the highest-scoring algorithm in the challenge leaderboard, it represents a good baseline because (1) it is among the top-performing published works, (2) it does not use task-dependent features, and (3) it can be easily adapted to other sequencing tasks.

Explanation Accuracy

As mentioned in the second chapter, it is hard to evaluate the goodness of an explanation. In STC settings, in fact, there isn't a background knowledge able to tell us which premise should be considered the best one to solve the example.

Recall that Ancona et al [5] propose to test the model on many versions of input, where a part of it is occluded, to extract the *surrogate ground truth*. Starting from this idea, there are produced, for each example, four versions of original input, each composed of a single premise. Then the model is tested on each input version to check whether, at least one of them, it produces a different output respect to the complete original input. In STC dataset, the described behavior is observed in ~10% of the samples.

Consider, also, that the proposed method generates an output similar to the one in Figure 5.2, thus allowing to identify the best and worst scoring explanations (or equivalently for this task, premises). Based on this, it is possible to check, when the previous condition is satisfied, if the model output confirms or contradicts the general prediction, when it is used, as input, only the best or the worst premise. Therefore the *explanation accuracy* tells us how many times the output obtained using only the selected premise matches the output using the complete input.

Intuitively, if the explanation for a certain prediction is good, feeding that information alone to the network should result in the same prediction (and a good accuracy). Conversely, by only providing the worst explanation to the model, the prediction should change due to the missing relevant information, leading to a low explanation accuracy.

Choice of the threshold

In the current settings, the only parameter that can change the output of the explanation module and that can be tuned, is the number of read cells N_c to consider at each step. Therefore, there are compared the explanation accuracy obtained using multiple values of N_c, to evaluate the relation between the explanation score, the obtained explanation accuracy and the number of most-read cells considered in the history.

Explanations Adequacy

Finally, the system has been verified against two recently introduced tests that check if an explanation method is invariant respect to the labels and to the learned parameters of the model [2].

5.3.3 Results

Baseline

The table 5.1 shows that the method and the baseline [51] obtain the same score on the test set and, consequently, they achieve the same generalization capabilities. The difference of dev accuracy can be explained by their extensive grid search over the architecture's parameters on the development set, and the picking up of the best model. The gap between dev and test set is a signal of such behavior. In this work instead, their configuration is used as a base and the grid search is applied mainly for the explanation module parameters. The only changed parameters respect the original configuration are the batch size set to 1 as in the original DNC paper and the number of cells inside the LSTM, that speed up the computation returning the same scores. Indeed, the objective is to translate the same approach on a MANN architecture and not to propose a better architecture configuration.

Additionally, to confirm that these results mostly depend on the DNC usage of the memory content, the model is further tested by setting the LSTM output to zero

Table 5.1. Avg. accuracy comparison against task baseline. For [51], the scores are reported from the original paper, while the system average accuracy is computed over 10 different runs.

Avg. Accuracy		
Architecture	Dev	Test
Mihaylov et al. [51]	77.12%	72.10%
DNC	71.30%	72.10%

and checking that the overall performance does not drop too much (68.8%). This confirms the importance of the memory content, and its relevance for explainability purposes.

Explanation Accuracy

The results presented in Table 5.2 contain the explanation accuracy obtained when using the best, the worst and a random selected premise (acting as a control variable).
Remember that the explanation accuracy tells us how many times the prediction returned using as input only the selected premise matches the prediction obtained using all the premises, and that the examples considered are those for which is possible to obtain a different prediction using one of the premises (~10% of the dataset). In the other cases, in fact, the accuracy would be 100%, due probably to its overabundant nature, and there isn't a way to extract the surrogate ground truth.

The results confirm the hypothesis. The best premise matches the general prediction more often than a random chosen premise in all the datasets, while the worst premise has a lower accuracy than the random one, again, in all the cases. All the trials are obtained by building the read history with the 10 most read cells at every step. It is significant also the gap between the best and worst explanation (premise) , quite stable and always greater than 10%, meaning that effectively exist a difference between the predictions obtained using the two premises.

Note that the worst premise can achieve a lower explanation accuracy than a random premise due to the structure of the dataset, that often contains a misleading or poorly relevant premise of the story. In other datasets and settings, the worst and the random premise should have the same scores.

In Figure 5.2 are shown four examples of explanation module outputs. In the first one the words of the most informative premise (*best premise*) , highlighted in blue, has the 48.3% of readings during the parsing of the first ending, while the third has no readings during that parsing and therefore is the less informative (*wors premise*)

Table 5.2. Avg. explanation accuracy over 10 runs on the Story Cloze Test when feeding the model with only a random, the best and the worst premise.

Avg. Explanation Accuracy			
Premise	Train	Dev	Test
Random	57.5%	60.0%	55.6%
Best	66.0%	73.3%	63.8%
Worst	51.3%	53.3%	53.6%

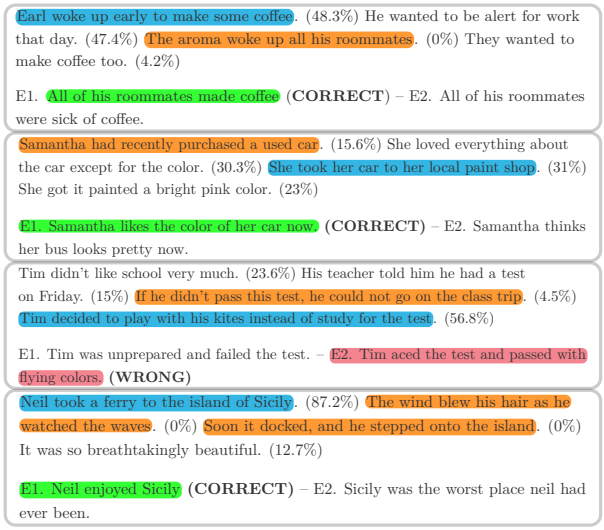

Figure 5.2. Example outputs on the Story Cloze Test. A relevance score is associated to each premise – ranked from best (blue) to worst (orange). Predictions are highlighted in green if correct, or red if wrong.

, and it is highlighted in orange. Note that if the prediction was the second ending, then the explanations would be different because, in that case, would be considered only the readings during the parsing of second ending.

Threshold impact on explanation accuracy

In Table 5.3 are shown several explanation accuracy values for different thresholds, where the notation $topN_c$ indicates the used N_c value. The system is tested considering $N_c \in \{1, 5, 10, 25\}$ in addition to the above-median case.

The results show that the explanation accuracy of both the best and the worst premises follows the same behavior: the smaller the value of N_c, the higher the explanation accuracy; the larger the threshold, the lower the accuracy.

The observed behavior can be explained by analyzing the information in the read

Table 5.3. Avg. explanation accuracy obtained on the dev set over 10 runs for the best and worst premises using different N_c values. N_c controls how many read cells are considered at each step to build the history. In bold are highlighted the best and worst performance respectively for each premise type among the different threshold values.

Dev Set Avg. Explanation Accuracy vs Thresholds					
	Threshold				
Premise	Top1	Top5	Top10	Top25	Above Median
Best	**75.6%**	**75.6%**	73.3%	71.1%	40.0%
Worst	60.0%	60.0%	53.3%	**40.0%**	48.9%

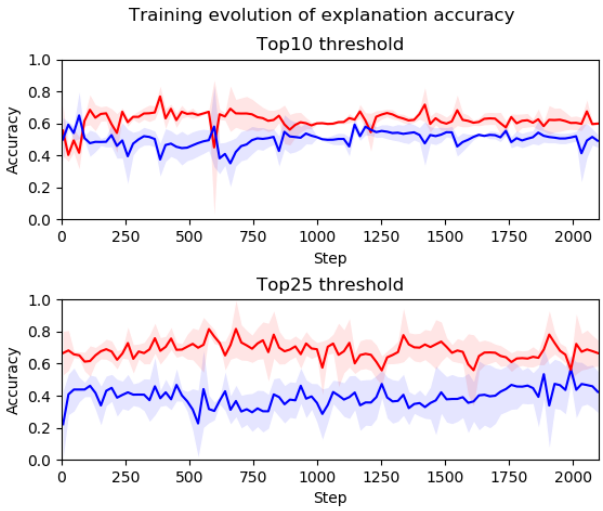

Figure 5.3. Training evolution of the explanation accuracy. The plot shows the mean explanation accuracy and its standard deviation obtained on the training set over 10 runs when using only the best (red line) and the worst (blue line) premise as input. Explanations are computed using a read history generated from the 10 and 25 most read cells at every step for the top and bottom figures respectively.

history. For example, when $N_c = 1$ (*top1* threshold) a single cell for each step is available in the history, which contains – at the end of the sequence – only T read cells, where T is the number of steps (i.e., words). T is generally quite small, and commonly stored cells only refer to one or two premises. Moreover, since these cells are those with the highest weight, they are highly represented in the read vector returned by the memory, and they have a direct influence on the final prediction. This allows us to conclude that these cells contain the best explanation for the current prediction.
The worst explanation case is more tricky: as described above, when using small N_c values, many premises do not have read cells in the history, and the explanation module has to (almost) randomly choose the worst premise. This lack of history information results in a large probability of picking a sufficiently good explanation instead of the bad one, due to the overabundant nature of the SCT dataset.

Conversely, when N_c is set to a large value (e.g., $N_c = 25$) , the generated history contains several cells. If no step of a certain sequence is contained in the history, the sequence itself is not represented in the read vector and it cannot be used in the final prediction. This allows us to identify the worst premise, and to significantly reduce the matching accuracy when feeding it to the network, as expected.
The choice of the best explanation, instead, is based on a larger number of cells, resulting in an increased selection noise. In fact, while a small N_c value rewards premises that are read more constantly during inference, a larger threshold only rewards premises with the greatest number of readings – independently of when the read occurred. Hence, premises contained in cells that are read multiple times only at the beginning are ranked higher than premises read fewer times but in a more constant way, lowering the accuracy.

Starting from these observations, can be concluded that N_c must be set depending on whether we are only interested to the most useful explanation or to a more informative analysis.

Finally, Figure 5.3 shows the evolution of the explanation accuracy for the best and the worst premises over 10 runs, confirming the results of Table 5.2. These plots additionally indicate that the explanations provided by the system are meaningful from the very first training steps and, consequently, that the network quickly learns to use its memory (independently of the final prediction).

Explanation Adequacy

Despite the lack of universal metrics, it is possible to try to study the adequacy of explanations by adapting the tests proposed by Adebayo et al. [2]. They show that visual inspection is a poor guide to judge an explanation for saliency maps methods. Instead, they propose two tests: model parameter randomization test and data randomization test.

The first compares the output of a saliency method on a randomly initialized untrained network and the output of the complete trained network. If the approach depends on the parameters learned by the network during the training process, then the two networks should produce considerably different results.
The latter creates a new training set, shuffling the label of the original training set, and then trains a new model on such modified dataset. Clearly, in this case, no predictive model can do better than random guessing, because the relationship between instances and labels is broken, especially when the number of classes is sufficiently high.

The tests have been adapted, in STC settings, in the following way: in the model parameter randomization test are compared the explanation accuracy of the best and the worst premises, when the model is untrained and the weights are randomly initialized. As expected the test accuracy of the model (Table 4) is equivalent to random guessing. Moreover, the explanation accuracy of best and worst premise is almost the same, meaning the method cannot differentiate between a good and a bad premise. This confirms that the proposed method depends on the learned parameters, and in particular on how the model learns to use the memory.

In the data randomization test, the labels are shuffled in the training and development set and a new model is trained on the modified dataset. As proposed by the

Table 5.4. Adequacy tests adapted from [2].

Avg. Explanation Accuracy			
Test	Accuracy	Best	Worst
Parameter Randomization	50.8%	56.0%	55.4%
Data Randomization	58.6%	51.8%	62.8%

authors [2], the training process is stopped when the training accuracy is above 95%. From the results reported in Table 4 you can note that accuracy is higher than random guessing. This behavior depends on the number of classes in SCT: because the model has only two labels, the half of the examples in dataset could have potentially the right labels, allowing the network to learn a little bit about the problem.
Note also that, in this case, the role of best and worst premises is flipped, and their performance is worse than before. The flipping behavior can be explained as an attempt by the model to exploit the changed label: in this case to predict the correct answer, it should choose the answer that has the minimum similarity with the memory output. In fact, since the task is overabundant, this behavior can also be rewarded by some of the correctly labeled examples.

5.4 Novelty and relation with other SOTA approaches

The novelties of the described approach are:

- dealing with *sequential data* naturally without any adaptation;
- *flexibility*, indeed there can be obtained different explanations tracking and collecting the information about the memory usage in different ways;
- *deterministic* because it doesn't involve random input perturbation;
- *fast and light* because it does not require additional computational and time resources – that are needed to train a new classifier or fit an external function.

Respect to the local surrogate methods, it doesn't need neither to generate synthetic data nor to fit a new model, therefore it is faster than them. In the considered settings, in particular for the STC case, LIME takes ~15min to return the explanation of a single example, against the few seconds needed by the explanation module to calculate its scores. The time required for LIME is influenced by the high variability of the input features, and the size of the input itself. Moreover, it depends on some customizable parameters, like the number of features to return and the number of samples that make the approach not deterministic for low and mid values of them. The same considerations are valid also for SHAP, especially on its kernel version. On the other side the applicability of the proposed approach is limited to the MANNs architectures, while LIME or SHAP are model-agnostic.

In Figure 5.4 you can see an example of LIME [57][1] application on STC, where there are highlighted in blue and orange the words that shift the prediction towards the first or the second ending respectively, if removed. The main problem is that it doesn't explicitly support sequences, that have to be provided all at once and that one of a basic assumption of LIME is the independence of features, that does not

[1]Note that SHAP and LIME feature similar issues.

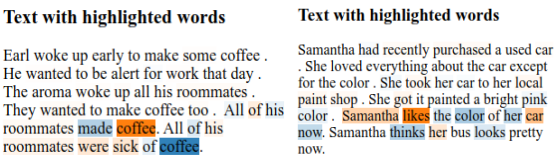

Figure 5.4. Example output of LIME, where each word in the sentence is weighted based on its relevance. Blue words, if removed, decrease the probability of choosing the first ending; orange words, if removed, decrease the probability of choosing the second ending.

hold for sequential problems.
As a result, LIME focuses its attention on the last part of the input (i.e., the concatenation of the premises and the answers), thus reporting on which part of the answer is more important. While it is true that changing the words of the answers leads to a different prediction, we are interested in knowing which premise or sequence of words in the story leads the model to output its prediction. In theory, the weights of the words in each premise could be summed. However, most of the times their weights are close to zero, and they change iteration by iteration so the explanations aren't stable enough.

The proposed method, conversely, exploits the sequential nature of the sentences, and checks which sub-sequence is more important. Figure 5.2 shows few instances of explanations obtained via the algorithm – including the ones in Figure 5.4 for direct comparison. Conversely, our approach is not suited to score the relevance of a single feature of a data point yet.

It appears similar to the approaches like LRP and DeepLIFT, in the sense that it is linked directly to the decision process of the network. While such approaches use the different values of activation functions or gradient to link the outcome to the input, these methods link the change in the usage of memory to both the input and output of the model.

Chapter 6

Conclusion

Nowadays, several researchers are focusing on making artificial intelligence and machine learning more explainable, with the purpose of increasing trust from the general public, and allowing usage in mission-critical domains, where experts supervision is mandatory. In this thesis, it is investigated how memory-augmented neural networks can be used to produce hints of explanation at prediction time.

The described approach represents an unexplored and novel way to collect information about the inference process. Yet, the results confirm that, by keeping track of the memory usage, it is possible produce reliable explanations on two different tasks. There are analyzed, for these tasks, the relation between the amount of tracked memory and the explanation goodness, as well as the gap between the best and the worst explanations.

These promising results make the proposed approach definitely worth additional investigation. In particular, there are several aspects that can be improved exploring new MANNs or studying new methodologies.
For example, in the current settings, the DNC output depends on both the memory contents and the controller output, meaning that sometimes the memory information can be ignored by the network and the explanations retrieved, in these cases, become useless. Shutting down completely the controller output lowers, as shown, the performance and it isn't an option. Could be studied, instead, an alternative architecture where there is a more direct correspondence between output and memory contents. A more linked relation should, also, improve the quality of explanation increasing the gap between the best and the worst premise in Story Cloze Test.

Another direction is the extension of the mechanism from sequential data to single-step prediction, making possible an extensive comparison between the proposed approach and agnostic ones like SHAP or LIME. This is not an easy task and maybe could be based on the exploiting the combination between attention mechanisms and external memory.

Finally, regarding the type of explanations, while in this thesis are retrieved mainly

prime implicants, the method can be used to produce different types. In tasks where the features are fixed, tracking the memory usage during the training process, could highlight what are the most important features and which values they assume to choose one class over another. Based on studies of this type there can be extracted also prototypes useful to eventually construct a counterfactual explanation.

Bibliography

[1] Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., and et al. (2016). Tensorflow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, OSDI'16, page 265–283, USA. USENIX Association.

[2] Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., and Kim, B. (2018). Sanity checks for saliency maps. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, page 9525–9536, Red Hook, NY, USA. Curran Associates Inc.

[3] Agarap, A. F. (2018). Deep learning using rectified linear units (relu). *ArXiv*, abs/1803.08375.

[4] Alvarez-Melis, D. and Jaakkola, T. S. (2018). On the robustness of interpretability methods. *ArXiv*.

[5] Ancona, M., Ceolini, E., Öztireli, C., and Gross, M. (2017a). Towards better understanding of gradient-based attribution methods for deep neural networks. *ICLR 2018 Sixth International Conference on Learning Representations*.

[6] Ancona, M., Ceolini, E., Öztireli, C., and Gross, M. (2017b). A unified view of gradient-based attribution methods for deep neural networks. *5th International Conference on Learning Representations (ICLR 2017)*.

[7] Augasta, M. G. and Kathirvalavakumar, T. (2011). Reverse engineering the neural networks for rule extraction in classification problems. *Neural Processing Letters*, 35:131–150.

[8] Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *ArXiv*.

[9] Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K.-R., and Samek, W. (2015). On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLOS ONE*, 10(7):e0130140.

[10] Bakker, B. (2002). Reinforcement learning with long short-term memory. In *Advances in neural information processing systems*, pages 1475–1482.

[11] Biran, O. and Cotton, C. (2017). Explanation and justification in machine learning: A survey. In *IJCAI 2017 Workshop on Explainable Artificial Intelligence (XAI)*.

[12] Bolukbasi, T., Chang, K.-W., Zou, J., Saligrama, V., and Kalai, A. (2016). Man is to computer programmer as woman is to homemaker? debiasing word embeddings. *NIPS'16: Proceedings of the 30th International Conference on Neural Information Processing Systems*.

[13] Boz, O. (2002). Extracting decision trees from trained neural networks. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining - KDD '02*. ACM Press.

[14] Buhrmester, V., Münch, D., and Arens, M. (2019). Analysis of explainers of black box deep neural networks for computer vision: A survey. *ArXiv*.

[15] Buçinca, Z., Lin, P., Gajos, K. Z., and Glassman, E. L. (2020). Proxy tasks and subjective measures can be misleading in evaluating explainable ai systems. *ArXiv*.

[16] Byrne, R. M. J. (2019). Counterfactuals in explainable artificial intelligence (XAI): Evidence from human reasoning. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization.

[17] Chipman, H. A., George, E. I., and Mcculloch, R. E. (1998). Making sense of a forest of trees. In *Proceedings of the 30th Symposium on the Interface*, pages 84–92.

[18] Clevert, D.-A., Unterthiner, T., and Hochreiter, S. (2015). Fast and accurate deep network learning by exponential linear units (elus). *ICLR 2016*.

[19] Craven, M. W. and Shavlik, J. W. (1994). Using sampling and queries to extract rules from trained neural networks. In *Machine Learning Proceedings 1994*, pages 37–45. Elsevier.

[20] Craven, M. W. and Shavlik, J. W. (1995). Extracting tree-structured representations of trained networks. In *NIPS*.

[21] Deng, H. (2018). Interpreting tree ensembles with inTrees. *International Journal of Data Science and Analytics*, 7(4):277–287.

[22] Dey, R. and Salemt, F. M. (2017). Gate-variants of gated recurrent unit (GRU) neural networks. In *2017 IEEE 60th International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE.

[23] Dhurandhar, A., Iyengar, V., Luss, R., and Shanmugam, K. (2017). Tip: Typifying the interpretability of procedures. *ArXiv*.

[24] Duchi, J., Hazan, E., and Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(null):2121–2159.

[25] Franke, J., Niehues, J., and Waibel, A. (2018). Robust and scalable differentiable neural computer for question answering. In *Proceedings of the Workshop on Machine Reading for Question Answering*, pages 47–59.

[26] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232.

[27] Fung, G., Sandilya, S., and Rao, R. B. (2005). Rule extraction from linear support vector machines. In *Proceeding of the eleventh ACM SIGKDD international conference on Knowledge discovery in data mining - KDD 05*. ACM Press.

[28] Gilpin, L. H., Bau, D., Yuan, B. Z., Bajwa, A., Specter, M., and Kagal, L. (2018). Explaining explanations: An overview of interpretability of machine learning. In *2018 IEEE 5th International Conference on Data Science and Advanced Analytics (DSAA)*. IEEE.

[29] Goldstein, A., Kapelner, A., Bleich, J., and Pitkin, E. (2015). Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics*, 24(1):44–65.

[30] Goodman, B. and Flaxman, S. (2017). European union regulations on algorithmic decision-making and a "right to explanation". *AI Magazine*, 38(3):50–57.

[31] Graves, A., Wayne, G., and Danihelka, I. (2014). Neural turing machines. *arXiv preprint arXiv:1410.5401*.

[32] Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., Colmenarejo, S. G., Grefenstette, E., Ramalho, T., Agapiou, J., et al. (2016). Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471.

[33] Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., and Pedreschi, D. (2018). A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5):1–42.

[34] Hausknecht, M. and Stone, P. (2015). Deep recurrent q-learning for partially observable mdps. In *2015 AAAI Fall Symposium Series*.

[35] Hochreiter, S. (1998). The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 06(02):107–116.

[36] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.

[37] Hooker, S., Erhan, D., Kindermans, P.-J., and Kim, B. (2018). A benchmark for interpretability methods in deep neural networks. *33rd Conference on Neural Information Processing Systems (NeurIPS 2019)*.

[38] Huang, Z., Xu, W., and Yu, K. (2015). Bidirectional lstm-crf models for sequence tagging. *ArXiv*.

[39] Ignatiev, A., Narodytska, N., and Marques-Silva, J. (2018). Abduction-based explanations for machine learning models. *The Thirty-Third AAAI Conference on Artificial Intelligence (AAAI-19)*.

[40] Johansson, U. and Niklasson, L. (2009). Evolving decision trees using oracle guides. In *2009 IEEE Symposium on Computational Intelligence and Data Mining*. IEEE.

[41] Johansson, U., Niklasson, L., and Knig, R. (2003). Rule extraction from trained neural networks using genetic programming.

[42] Karim, A. and Zhou, S. (2015). X-TREPAN : A multi class regression and adapted extraction of comprehensible decision tree in artificial neural network. In *Computer Science & Information Technology (CS & IT)*. Academy & Industry Research Collaboration Center (AIRCC).

[43] Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. *ArXiv*.

[44] Krishnan, R., Sivakumar, G., and Bhattacharya, P. (1999). Extracting decision trees from trained neural networks. *Pattern Recognition*, 32(12):1999–2009.

[45] Lipton, Z. C. (2018). The mythos of model interpretability. *Queue*, 16(3):31–57.

[46] Lou, Y., Caruana, R., and Gehrke, J. (2012). Intelligible models for classification and regression. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining - KDD '12*. ACM Press.

[47] Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, pages 4765–4774.

[48] Manning, C., Surdeanu, M., Bauer, J., Finkel, J., Bethard, S., and McClosky, D. (2014). The stanford corenlp natural language processing toolkit. In *Proceedings of 52nd annual meeting of the association for computational linguistics: system demonstrations*, pages 55–60.

[49] Martens, D., , and and, F. P. (2014). Explaining data-driven document classifications. *MIS Quarterly*, 38(1):73–99.

[50] McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133.

[51] Mihaylov, T. and Frank, A. (2017). Story cloze ending selection baselines and data examination. *LSDSem 2017*, page 87.

[52] Mikolov, T., Yih, W.-t., and Zweig, G. (2013). Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 746–751.

[53] Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, 267:1–38.

[54] Mostafazadeh, N., Chambers, N., He, X., Parikh, D., Batra, D., Vanderwende, L., Kohli, P., and Allen, J. (2016). A corpus and cloze evaluation for deeper understanding of commonsense stories. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 839–849, San Diego, California. Association for Computational Linguistics.

[55] Mostafazadeh, N., Roth, M., Louis, A., Chambers, N., and Allen, J. (2017). Lsdsem 2017 shared task: The story cloze test. In *Proceedings of the 2nd Workshop on Linking Models of Lexical, Sentential and Discourse-level Semantics*, pages 46–51.

[56] Núñez, H., Angulo, C., and Català, A. (2002). Rule extraction from support vector machines. In *ESANN*.

[57] Ribeiro, M., Singh, S., and Guestrin, C. (2016). "why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*. Association for Computational Linguistics.

[58] Ribeiro, M. T., Singh, S., and Guestrin, C. (2018). Anchors: High-precision model-agnostic explanations. In *AAAI Conference on Artificial Intelligence (AAAI)*.

[59] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.

[60] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.

[61] Samek, W., Binder, A., Montavon, G., Lapuschkin, S., and Muller, K.-R. (2017). Evaluating the visualization of what a deep neural network has learned. *IEEE Transactions on Neural Networks and Learning Systems*, 28(11):2660–2673.

[62] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2019). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*.

[63] Shih, A., Choi, A., and Darwiche, A. (2018). A symbolic approach to explaining bayesian network classifiers. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization.

[64] Shrikumar, A., Greenside, P., and Kundaje, A. (2017). Learning important features through propagating activation differences. *ICML'17: Proceedings of the 34th International Conference on Machine Learning*.

[65] Shrikumar, A., Greenside, P., Shcherbina, A., and Kundaje, A. (2016). Not just a black box: Learning important features through propagating activation differences. *ArXiv*.

[66] Simonyan, K., Vedaldi, A., and Zisserman, A. (2013). Deep inside convolutional networks: Visualising image classification models and saliency maps. *ICLR 2013*.

[67] Sokol, K. and Flach, P. (2018). Glass-box: Explaining AI decisions with counterfactual statements through conversation with a voice-enabled virtual assistant. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization.

[68] Springenberg, J. T., Dosovitskiy, A., Brox, T., and Riedmiller, M. A. (2014). Striving for simplicity: The all convolutional net. *CoRR*, abs/1412.6806.

[69] Srinivasan, S., Arora, R., and Riedl, M. (2018). A simple and effective approach to the story cloze test. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, volume 2, pages 92–96.

[70] Sukhbaatar, S., Szlam, A., Weston, J., and Fergus, R. (2015). End-to-end memory networks. *NIPS 2015*.

[71] Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic attribution for deep networks. *ICML'17: Proceedings of the 34th International Conference on Machine Learning*.

[72] Tan, S., Soloviev, M., Hooker, G., and Wells, M. T. (2016). Tree space prototypes: Another look at making tree ensembles interpretable. *ArXiv*.

[73] Wachter, S., Mittelstadt, B., and Russell, C. (2017). Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *SSRN Electronic Journal*.

[74] Wierstra, D., Foerster, A., Peters, J., and Schmidhuber, J. (2007). Solving deep memory pomdps with recurrent policy gradients. In *International Conference on Artificial Neural Networks*, pages 697–706. Springer.

[75] Zeiler, M. D. and Fergus, R. (2014). Visualizing and understanding convolutional networks. In *European Conference on Computer Vision - ECCV 2014*, pages 818–833. Springer International Publishing.

[76] Zhou, B., Khosla, A., Lapedriza, A., Oliva, A., and Torralba, A. (2016). Learning deep features for discriminative localization. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE.